1 Basi della programmazine in C

1.1 Dimensioni e Classificazione dei Linguaggi di Programmazione

- **Livello di Astrazione (Basso livello vs Alto livello):**
  - Nei linguaggi a basso livello (come l'Assembly), il set di istruzioni impiegato per la computazione è estremamente semplice e prossimo all'architettura hardware della macchina.
  - Nei linguaggi ad alto livello (come C e Java), le primitive sono significativamente più ricche, articolate ed astratte rispetto al funzionamento fisico del processore.
- **Campo di Applicazione (Generale vs Mirato):**
  - Nei linguaggi di tipo generale (general-purpose), l'insieme delle primitive consente lo sviluppo di un'ampia gamma di applicazioni eterogenee.
  - I linguaggi mirati (targeted) sono concepiti per domini applicativi specifici; un esempio è MATLAB (matrix laboratory), progettato appositamente per il calcolo numerico e le operazioni su matrici e vettori.
- **Modalità di Esecuzione (Interpretato vs Compilato):**
  - Nei linguaggi interpretati (come Python), il codice sorgente viene letto ed eseguito direttamente a runtime dall'interprete, il quale controlla il flusso valutando le istruzioni riga per riga.
  - Nei linguaggi compilati (come C), il codice sorgente deve essere preventivamente tradotto da un compilatore in codice oggetto/binario nativo prima di poter essere eseguito.
- **Paradigmi di Programmazione:**
  - *Programmazione Funzionale:* Modella il calcolo come valutazione di funzioni matematiche pure (es. Lisp, Scheme, Haskell).
  - *Programmazione Imperativa:* Descrive la computazione attraverso sequenze di istruzioni che modificano lo stato interno del programma (es. FORTRAN, BASIC, Pascal, C).
  - *Programmazione Logica/Dichiarativa:* Esprime la logica del calcolo senza descriverne esplicitamente il flusso di controllo operativo (es. Prolog).
  - *Programmazione Orientata agli Oggetti (OOP):* Impiega "oggetti" — strutture dati composte da campi dati e metodi associati — e le loro interazioni per progettare applicazioni (es. C++, Java, C#, Python).

1.2 Contesto Storico e Caratteristiche di C e Python

- **Linguaggio C:**
  - Sviluppato nel 1973 da Ken Thompson e Dennis Ritchie presso i laboratori AT&T Bell Labs per la realizzazione del sistema operativo UNIX.
  - È diffuso ed utilizzato su praticamente qualsiasi sistema operativo moderno ed è il linguaggio d'elezione per la scrittura di software di sistema.
  - Possiede un linguaggio core estremamente minimale e semplice, delegando le funzionalità non essenziali a librerie standard regolamentate.
  - Garantisce l'accesso a basso livello alla memoria fisica del computer mediante l'uso esplicito dei puntatori.
  - Ha rappresentato il fondamentale predecessore linguistico per C++, C# e Java.
- **Linguaggio Python:**
  - Creato da Guido van Rossum verso la fine degli anni '80.
  - Supporta nativamente molteplici paradigmi di programmazione: orientato agli oggetti, strutturato e funzionale.
  - Utilizza la tipizzazione dinamica e la gestione automatica della memoria tramite Garbage Collection.

1.3 Compilatori vs Interpreti e Processo di Sviluppo in C

- La differenza sostanziale tra C e Python risiede nelle modalità di esecuzione dei programmi scritti nei rispettivi linguaggi.
- L'interprete legge il codice sorgente scritto dall'utente ed esegue direttamente le operazioni istruzione per istruzione.
- Il compilatore non esegue direttamente il programma; genera invece un file contenente la traduzione del codice sorgente nel linguaggio macchina nativo.
- Il processo richiede prima l'esecuzione del compilatore per produrre il file binario eseguibile e, successivamente, l'avvio indipendente dell'eseguibile generato.
- Il ciclo di sviluppo standard in ambiente UNIX/Linux con il compilatore `gcc` si articola in tre fasi cicliche:
  1. *Creazione/Modifica:* Scrittura del file sorgente (es. `my_program.c`).
  2. *Compilazione:* Invocazione del compilatore mediante il comando `gcc my_program.c`, che genera il file binario `a.out`.

3. *Esecuzione:* Comando `./a.out` per ordinare al computer di eseguire il codice macchina generato.

- Durante l'esecuzione di `a.out`, l'elaboratore non conserva alcuna memoria del fatto che il file sia stato generato a partire da un programma C.
- Il ciclo di sviluppo si arresta unicamente quando il programmatore si ritiene soddisfatto della correttezza funzionale e delle prestazioni del programma.
- La compilazione impone vincoli rigidi sul design del linguaggio: il C è progettato affinché il compilatore possa dedurre tutte le informazioni necessarie alla traduzione senza dover eseguire il codice.

1.4 Dichiarazione delle Variabili

- Il linguaggio C richiede la dichiarazione esplicita di ogni variabile prima del suo effettivo utilizzo, informando il compilatore sul tipo di dato associato.
- In Python non esiste il concetto di dichiarazione esplicita del tipo di variabile.
- Sintassi di dichiarazione: tipo di dato, spazio in bianco, nome della variabile e punto e virgola finale (es. `double x;`).
- L'assegnamento di un valore intero a una variabile dichiarata di tipo `double` (es. `x = 3;`) determina il salvataggio interno del valore in formato floating-point (`3.0`).
- Una volta dichiarata una variabile con un determinato tipo, non è più possibile modificarne il tipo all'interno del programma.
- Nel C tradizionale (standard C89), le dichiarazioni delle variabili devono risiedere obbligatoriamente all'inizio della funzione prima delle istruzioni operative.
- Se si tenta di utilizzare una variabile non dichiarata, il compilatore rifiuta di compilare il programma segnalando un errore.

1.5 Gestione dello Spazio in Bianco (Whitespace)

- In Python gli spazi in bianco (tabulazioni e nuove linee) hanno valore sintattico determinante per la separazione delle istruzioni e per l'indentazione dei blocchi di codice (es. corpi di `if` o `while`).
- In C gli spazi in bianco sono del tutto irrilevanti e servono esclusivamente a separare le parole chiave o i token.

- Le istruzioni in C sono terminate dal punto e virgola (`;`) e la portata dei blocchi di codice è delimitata dalle parentesi graffe ({}).
- È sintatticamente ammissibile per il compilatore scrivere un intero programma C su un'unica riga continua, ma costituisce una pessima pratica di programmazione per la leggibilità.

1.6 Output Formattato con la Funzione `printf()`

- La stampa a video in C viene effettuata mediante la funzione `printf()`, fornita dalla libreria standard `<stdio.h>`.
- Il primo argomento della funzione `printf()` è una stringa di formato, seguita dall'elenco dei valori da stampare associati ai relativi segnaposto.
- Il carattere percentuale (`%`) è speciale: indica che nel punto corrispondente verrà inserito il valore del parametro successivo.
- Specificatori di formato supportati:
  - `%d`: stampa valori interi (`int`) in forma decimale.
  - `%f`: stampa valori reali (`double` / `float`) in notazione decimale.
  - `%e`: stampa valori reali (`double`) in notazione scientifica (es. `3.000000e8`).
  - `%c`: stampa un singolo carattere (`char`).
  - `%s`: stampa una sequenza o stringa di caratteri.
- Sequenze di escape valide all'interno delle stringhe C:
  - `\n`: carattere di nuova linea (newline).
  - `\t`: carattere di tabulazione (tab).
  - `\"`: carattere virgolette doppie.
  - `\\`: carattere barra rovesciata (backslash).
- Le sequenze di escape fanno parte della sintassi nativa del linguaggio C e non rappresentano una caratteristica esclusiva della funzione `printf()`.
- Le chiamate successive alla funzione `printf()` vengono eseguite dal computer in modo strettamente sequenziale.

1.7 Struttura e Definizione delle Funzioni in C

- A differenza di Python, tutto il codice eseguibile in C deve essere racchiuso all'interno di funzioni.
- Le funzioni in C non possono essere annidate l'una all'interno dell'altra.
- La struttura di un programma C consiste in un elenco ordinato di definizioni di funzioni, eseguite su chiamata.
- Definizione sintattica di una funzione: specificazione del tipo di ritorno, nome della funzione, parentesi tonde contenenti l'elenco dei parametri (ciascuno preceduto dal proprio tipo) e corpo della funzione racchiuso tra parentesi graffe.
- Se una funzione non restituisce alcun valore utile, viene utilizzato il tipo di ritorno `void`.
- Ogni programma C deve contenere la funzione speciale `main()`, che costituisce l'unico punto di avvio (entry point) dell'esecuzione.
- La funzione `main()` ha come tipo di ritorno un intero (`int`); per convenzione si restituisce il valore `0` per segnalare la corretta conclusione del programma.

1.8 Operatori e Gerarchia delle Precedenze

- **Precedenza degli Operatori in C (dalla massima alla minima):**
  1. Postfisso: `++`, `--`
  2. Unario: `+`, `-`, `!`
  3. Moltiplicativo: `*`, `/`, `%`
  4. Additivo: `+`, `-`
  5. Relazionale: `<`, `>`, `<=`, `>=`
  6. Uguaglianza: `==`, `!=`
  7. AND Logico: `&&`
  8. OR Logico: `||`
  9. Assegnamento: `=`, `+=`, `-=`, `*=`, `/=`, `%=`
- **Confronto ed Eccezioni Rispetto a Python:**
  - Il C non dispone di un operatore per l'elevamento a potenza equivalente a `**` di Python; le potenze si calcolano con la funzione di libreria `pow(base, esponente)` (es. `pow(1.1, 2.0)`).

- Le operazioni booleane utilizzano simboli algebrici (`&&`, `||`, `!`) al posto delle parole chiave inglesi (`and`, `or`, `not`).
- L'operatore di NOT (`!`) ha un livello di precedenza estremamente elevato, rendendo quasi sempre necessarie le parentesi tonde.
- In C l'assegnamento `=` è un operatore che restituisce il valore assegnato, mentre in Python è un'istruzione.
- L'assegnamento in C può far parte di un'espressione complessa (es. `while ((a = getchar()) != EOF)`), consentendo di assegnare il valore restituito e valutarlo contestualmente.
- L'annidamento dell'assegnamento all'interno di condizioni è considerato da molti programmatori una cattiva pratica di stile, sebbene ritenuto comodo da altri; in Python tale costrutto è vietato sintatticamente.
- Gli operatori unari `++` e `--` incrementano o decrementano il valore di una variabile di un'unità (es. `i++` equivale a `i = i + 1`).
- L'operatore di divisione `/` esegue la divisione intera se entrambi gli operandi sono interi, troncando l'eventuale resto (es. `13 / 5` restituisce `2`, mentre `13 / 5.0` restituisce `2.6`).

1.9 Tipi di Dato Fondamentali, Modificatori e Logica Booleana

- Tipi primitivi base: `int` (numeri interi), `char` (singolo carattere ASCII), `float` (virgola mobile a singola precisione), `double` (virgola mobile a doppia precisione).
- Modificatori di tipo applicabili: `signed`, `unsigned`, `short`, `long`.

- **Gestione della Logica Booleana (Standard C89):**
  - Nel C tradizionale non esiste un tipo booleano nativo dedicato.
  - L'intero `0` viene valutato come `falso`, mentre qualsiasi valore intero diverso da zero viene interpretato come `vero`.
  - Gli operatori relazionali e logici restituiscono numeri interi: `1` per indicare vero e `0` per indicare falso.
  - Tale caratteristica consente di scrivere scorciatoie sintattiche come `pos = (a > 0) + (b > 0) + (c > 0);`, la quale calcola quanti valori tra a, b e c sono strettamente positivi.
  - I programmatori esperti considerano questo comportamento un limite del linguaggio ed evitano le scorciatoie, preferendo il confronto esplicito con lo zero (es. `x != 0`) per garantire chiarezza di stile.

Tabella 1: Specifiche e Intervalli dei Tipi di Dato Fondamentali in C

Tipo di Dato	**Occupazione di Memoria**	**Intervallo di Valori Ammissibili (Range)**
`signed char`	1 Byte	Da -127 a $+127$
`unsigned char`	1 Byte	Da 0 a 255
`short int`	2 Byte	Da -32.767 a $+32.767$
`unsigned short int`	2 Byte	Da 0 a 65.535
`int`	4 Byte (su Unix)	Da -32.767 a $+32.767$ (minimo garantito); fino a $-2^{31} \dots 2^{31} - 1$
`unsigned int`	4 Byte	Da 0 a 65.535 (o fino a $2^{32} - 1$)
`long int`	8 Byte	Da $-2.147.483.647$ a $+2.147.483.647$
`unsigned long int`	8 Byte	Da 0 a 4.294.967.295
`float`	4 Byte	Da $\sim 10^{-37}$ a $\sim 10^{38}$
`double`	8 Byte	Da $\sim 10^{-307}$ a $\sim 10^{308}$
`long double`	16 Byte	Da $\sim 10^{-4931}$ a $\sim 10^{4932}$

1.10 Regole delle Parentesi Graffe e Struttura `else if`

- Il corpo delle strutture di controllo (come `if`) racchiude le istruzioni mediante parentesi graffe ({}).
- Se il corpo di un ramo racchiude un'unica istruzione, l'uso delle parentesi graffe è opzionale.
- Poiché l'intero blocco `if...else` conta come un'unica istruzione sintattica, la clausola `else` può contenere un altro `if` senza richiedere graffe aggiuntive.
- Questa regola consente di concatenare più verificate sintattiche in sequenza (`else if`) senza la necessità di introdurre una parola chiave specifica come l' `elif` presente in Python.

1.11 Tipologie e Strutture delle Istruzioni (Statements)

1. **Dichiarazione di Variabile:** Istruzione propria del C priva di parallelo in Python (es. `int x;`).

2. **Espressione come Istruzione:** Può consistere in un operatore che modifica lo stato di una variabile (`x = 3;`, `x += 1;`, `x++;`) o in una chiamata a funzione (es. `printf("%d", x);`).

3. **Istruzione `if`:** La condizione deve risiedere tra parentesi tonde, non è presente alcun due punti al termine, e il corpo è delimitato da graffe.

4. **Istruzione `return`:** Provoca l'uscita immediata dalla funzione restituendo un valore (oppure `return;` in funzioni con tipo di ritorno `void`).

5. **Istruzione `while`:** Funziona in modo analogo al corrispettivo Python, richiedendo le parentesi tonde per la condizione.

6. **Istruzione `for`:**

   - Sintassi formale: `for (inizializzazione; test; aggiornamento) corpo;`.
   - L'espressione di *inizializzazione* viene eseguita una sola volta prima dell'avvio del ciclo.
   - L'espressione di *test* viene valutata prima di ogni iterazione; il ciclo procede finché la condizione risulta vera (diversa da zero).
   - L'espressione di *aggiornamento* viene eseguita al termine di ogni esecuzione del corpo della funzione.
   - Esempio di conteggio sequenziale: `for (i = 0; i < n; i++)`.
   - Esempio di incremento geometrico: `for (p = 1; p <= 512; p *= 2)`.

1.12 Strutture Dati Composite: Array e Gestione della Memoria

- L'array costituisce l'unica struttura dati composita nativa fondamentale analizzata in questo capitolo.

- A differenza delle liste Python, la dimensione dell'array in C è definita rigidamente al momento della creazione e non può essere ridimensionata.

- Sintassi di allocazione: `double pops[50];` oppure con valori definiti `char vowels[6] = {'a', 'e', 'i', 'o', 'u', 'y'};`.

- Il linguaggio C non fornisce alcun supporto o funzione integrata (come `len()`) per determinare la lunghezza di un array a runtime.

- **Mancanza del Controllo sui Limiti (Array-Bounds Checking):**

  – Se si tenta di accedere a un indice situato fuori dai limiti stabiliti (es. `pops[50]` o `pops[-100]`), il programma non segnala un errore in fase di compilazione ma esegue ciecamente la lettura o scrittura in memoria.

- Questa operazione va a sovrascrivere variabili adiacenti allocate nello stack.
- Esempio pratico di bug grave: qualora la variabile del ciclo `i` venga allocata in memoria subito dopo un array `vals[5]`, un ciclo che tenta di azzerare la posizione `vals[5]` azzera involontariamente la variabile `i`, resettando il ciclo all'infinito e impedendo il raggiungimento della fine del programma.
- I bug provocati dalla sovrascrittura accidentale della memoria sono estremamente difficili da individuare e richiedono lungo tempo per la riparazione.
- Il tentativo di accedere a locazioni di memoria non valide o protette causa un blocco improvviso dell'applicazione con la dicitura generica di sistema **"segmentation fault"**.

1.13 Sintassi dei Commenti

- **Commenti multilinea (Standard C originale):** Iniziano con la sequenza `/*` e terminano con `*/`; possono estendersi su più righe di codice.
- **Commenti su riga singola:** Introdotti originariamente dal C++ e supportati dai compilatori C moderni, iniziano con la doppia barra `//` e terminano alla fine della riga corrente.

1.14 Organizzazione del Codice: Prototipi, File Header e Preprocessore

- **Prototipi di Funzione:**
  - Poiché il compilatore analizza il codice dall'alto verso il basso, una funzione deve essere dichiarata prima di essere richiamata.
  - La prototipazione consente di specificare la sola intestazione della funzione seguita dal punto e virgola senza definire il corpo (es. `int gcd(int a, int b);`).
  - Segnala al compilatore che la funzione verrà definita successivamente, consentendo la validazione della sintassi della chiamata.
- **File Header (`.h`):**
  - Nei programmi suddivisi su più file sorgente, i prototipi condivisi vengono raggruppati in file di intestazione con estensione `.h`.
  - La direttiva `#include` ordina al preprocessore di sostituire l'istruzione con l'intero testo contenuto nel file header indicato.
  - Le parentesi angolari (es. `#include <stdio.h>`) indicano file d'intestazione standard forniti dalla libreria di sistema.

  - Le virgolette doppie (es. `#include "mathfun.h"`) indicano file header personalizzati presenti nella medesima cartella del sorgente.

- **Costanti e Direttiva `#define`:**
  - La direttiva `#define` istruisce il preprocessore ad eseguire una sostituzione testuale automatica all'interno del sorgente prima di avviare la compilazione.
  - Esempio: la direttiva `#define PI 3.14159` sostituisce ogni occorrenza dell'identificatore `PI` con la costante testuale `3.14159`.

2 Puntatori e Aritmetica dei Puntatori

2.1 Informazioni Amministrative e Logistica del Corso (Administrivia)

2.1.1 Organizzazione Didattica e Personale

- Il corso CSE 333 (Autumn 2025) è tenuto dagli istruttori Naomi Alterman e Chris Thachuk.
- Il team di Teaching Assistants (TA) comprende: Ann Baturytski, Derek de Leuw, Blake Diaz, Rishabh Jain, Chendur Jel Jayavelu, Lucas Kwan, Irene Xin Jie Lau, Nathan Li, Maya Odenheim, Advay Patil, Selim Saridede, Deeksha Vatwani, Angela Wu e Jiexiao Xu.
- L'Esercizio 2 (**EX 2**) è stato pubblicato e la consegna è fissata per mercoledì alle ore 10:00.
- L'Homework 0 (**HW0**) scade mercoledì notte alle ore 23:59.

2.1.2 Flusso di Lavoro Git e Verifica delle Consegne

- È obbligatorio utilizzare gli strumenti `cpplint` (per lo stile del codice) e `valgrind` (per la verifica della memoria) sia per gli homework che per gli esercizi.
- La procedura di sottomissione tramite Git richiede di eseguire nell'ordine: `git add`, `git commit`, `git push`, creare il tag con `git tag hw0-final` e infine inviare il tag al server remoto con `git push --tags`.
- **Procedura di verifica rigorosa**: Dopo aver inviato il tag, clonare il repository in una directory totalmente differente, effettuare il checkout del tag (`git checkout hw0-final`) e testare che tutto funzioni prima delle 23:59.
- Non limitarsi a controllare la pagina web di GitLab; la verifica reale va condotta clonando e testando in locale.

- In caso di problemi nel clone separato, eliminare la copia di test, correggere gli errori nel repository originale, rifare commit, push, tag e ripetere il controllo.
- Tutti gli esercizi e homework devono compilare ed eseguire correttamente sulle macchine Linux aggiornate della Allen School (`attu`, macchine di laboratorio, VM).

2.1.3 Criteri di Valutazione e Utilizzo di Gradescope

- La valutazione degli esercizi su Gradescope si basa su un giudizio globale espresso con punteggi ordinali: 3 (*superior*), 2 (*good*), 1 (*marginal*), 0 (*not sufficient for credit*).
- Ci si aspetta una prevalenza di voti pari a 2 all'inizio del corso, con un incremento di voti pari a 3 man mano che gli studenti acquisiscono esperienza.
- Gli elementi di rubrica contrassegnati con ± 0 sono utilizzati unicamente come strumento di comunicazione per fornire feedback e motivazioni sul voto.
- Le annotazioni ± 0 non alterano in alcun modo il conteggio dei punti finali.

2.2 Puntatori e Aritmetica dei Puntatori (Pointers & Pointer Arithmetic)

2.2.1 Diagrammi Box-and-Arrow e Rappresentazione dello Stack

- I diagrammi Box-and-Arrow illustrano la disposizione della memoria nello *stack frame* (es. della funzione `main()`) associando a ogni variabile tre componenti: indirizzo di memoria (*address*), nome della variabile (*name*) e valore contenuto (*value*).
- Nel programma `boxarrow.c`:
  - `int x = 1;` assegna valore 1 alla variabile scalare `x`.
  - `int arr[3] = {2, 3, 4};` alloca tre interi in posizioni di memoria contigue.
  - `int* p = &arr[1];` memorizza all'interno del puntatore `p` l'indirizzo dell'elemento `arr[1]`.
- L'ispezione della memoria tramite `printf` visualizza gli indirizzi esadecimali (es. `0x7fff...64` per `x`, `0x7fff...70` per `arr[0]`, `0x7fff...74` per `arr[1]`, `0x7fff...78` per `arr[2]`) e i relativi contenuti.

2.2.2 Tipizzazione, Puntatore Generico (`void*`) e Operazioni Valide

- I puntatori in C sono dotati di un tipo specifico, il quale comunica al compilatore la dimensione del dato puntato (sizeof($*p$)).
- **Eccezione (`void*`)**: Il tipo `void*` rappresenta un puntatore generico (un segnaposto).
- Poiché il tipo `void` non ha una dimensione definita, l'aritmetica dei puntatori non è applicabile a un `void*`.
- **Operazioni d'aritmetica dei puntatori valide**:
  1. Somma o sottrazione tra un puntatore e un numero intero (il cui valore viene scalato per sizeof($*p$)).
  2. Sottrazione tra due puntatori, valida esclusivamente se entrambi puntano all'interno del medesimo stack frame o dello stesso blocco allocato dinamica mediante `malloc`.
  3. Confronti tra puntatori ($<, \leq, ==, !=, >, \geq$), incluso il confronto con il valore `NULL`.

2.2.3 Tracciamento di Codice: Puntatori a Puntatori (`boxarrow2.c`)

- Si consideri il seguente codice di esempio:

```
int arr[3] = {2, 3, 4};
int* p = &arr[1];
int** dp = &p; // puntatore a puntatore

*(*dp) += 1;    // Istruzione 1
p += 1;         // Istruzione 2
*(*dp) += 1;    // Istruzione 3
```

- **Stato Iniziale**: `arr` = $\{2, 3, 4\}$, `p` punta ad `arr[1]` (indirizzo `0x...74`, valore 3), `dp` punta a `p` (indirizzo `0x...68`).
- **Esecuzione Istruzione 1** (`*(*dp) += 1`): La prima dereferenziazione `*dp` restituisce `p`; la seconda dereferenziazione accede ad `arr[1]` incrementandone il valore da 3 a 4. L'array diventa $\{2, 4, 4\}$.
- **Esecuzione Istruzione 2** (`p += 1`): L'incremento del puntatore fa avanzare `p` dell'ampiezza di 1 × sizeof(`int`) (4 byte). Ora `p` punta all'indirizzo `0x...78` (`arr[2]`).
- **Esecuzione Istruzione 3** (`*(*dp) += 1`): `*dp` valuta il nuovo valore di `p` (indirizzo di `arr[2]`); la dereferenziazione incrementa `arr[2]` da 4 a 5.

- **Valori finali dell'array `arr[]`**: $\{2, 4, 5\}$.

2.2.4 Rappresentazione dei Dati in Memoria: Endianness

- La memoria è indirizzata al singolo byte; l'endianness definisce l'ordine sequenziale con cui i byte di dati multi-byte vengono memorizzati e letti.
- **Big-Endian**: Il byte meno significativo (*Least Significant Byte* – LSB) risiede all'indirizzo di memoria più alto (e il byte più significativo MSB all'indirizzo più basso).
- **Little-Endian**: Il byte meno significativo (*Least Significant Byte* – LSB) risiede all'indirizzo di memoria più basso.
- **Esempio**: Memorizzazione dell'intero a 4 byte `0xa1b2c3d4` a partire dall'indirizzo `0x100`:
  - In *Big-Endian*: `0x100: a1`, `0x101: b2`, `0x102: c3`, `0x103: d4`.
  - In *Little-Endian*: `0x100: d4`, `0x101: c3`, `0x102: b2`, `0x103: a1`.

2.2.5 Esempio di Avanzamento Differenziato: `int*` vs `char*` (`pointerarithmetic.c`)

- Si analizzi l'esecuzione su architettura a 64-bit x86-64 (Little-Endian):

```
int arr[3] = {1, 2, 3};
int* int_ptr = &arr[0];
char* char_ptr = (char*) int_ptr;
```

- Inizialmente, `arr[0]` occupa 4 byte con rappresentazione esadecimale `01 00 00 00` all'indirizzo `0x...10`.
- `int_ptr += 1;`: Incrementa l'indirizzo di $1 \times$ sizeof(`int`) $= 4$ byte, spostandosi a `0x...14` (`arr[1]`). Il valore `*int_ptr` vale 2.
- `int_ptr += 2;`: Incrementa l'indirizzo di $2 \times 4 = 8$ byte, arrivando a `0x...1C`. Il puntatore esce dai confini dell'array e `*int_ptr` produce un valore sconosciuto/non definito (`???`).
- `char_ptr += 1;`: Incrementa l'indirizzo di soli $1 \times$sizeof(`char`) $= 1$ byte, passando da `0x...10` a `0x...11`.
  - All'indirizzo `0x...10`, `*char_ptr` legge il primo byte di `arr[0]`, ovvero `1`.
  - All'indirizzo `0x...11`, `*char_ptr` legge il secondo byte del numero 1 in Little-Endian, ovvero `0`.
- `char_ptr += 2;`: Incrementa l'indirizzo di 2 byte arrivando a `0x...13` (quarto byte di `arr[0]`), dove `*char_ptr` è ancora pari a `0`.

2.3 Puntatori come Parametri di Funzioni (Pointers as Parameters)

2.3.1 Meccanismo di Passaggio per Valore (Call-By-Value)

- Nel linguaggio C (analogamente a Java), gli argomenti delle funzioni vengono passati **esclusivamente per valore**.
- La funzione chiamata (*callee*) riceve una copia locale dei parametri allocata nei registri del processore o nel proprio stack frame.
- Eventuali modifiche effettuate dalla funzione chiamata sui propri parametri locali non alterano in alcun modo i valori memorizzati nelle variabili della funzione chiamante (*caller*).

2.3.2 Analisi Comparativa: Broken Swap vs. Fixed Swap

- **Broken Swap** (`brokenswap.c`):

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

  Quando il `main` esegue `swap(a, b)` con `a = 42` e `b = -7`, lo stack frame di `swap` alloca copie locali di `a`, `b` e `tmp`. Le modifiche invertono solo le copie locali (`a = -7`, `b = 42`). Alla chiusura di `swap`, il suo frame viene rimosso e le variabili del `main` rimangono inalterate (42 e -7).

- **Fixed Swap** (`swap.c`):

```
void swap(int* a, int* b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

  Invocando la funzione tramite `swap(&a, &b)`, la funzione `swap` riceve per valore gli indirizzi di memoria delle variabili del `main`. Dereferenziando i puntatori (`*a`, `*b`), la funzione agisce direttamente sullo stack frame del `main`, scambiando con successo i valori originali (`a` diventa -7, `b` diventa 42).

- **Layout della memoria di processo**: Include il Kernel OS (protegno), Stack (crece verso il basso), Heap (cresce verso l'alto), Read/Write Segment (`.data`, `.bss`), Read-Only Segment (`.text`, `.rodata`).

2.3.3 Parametri di Output (Output Parameters) e Regole d'Uso

- Un **parametro di output** è un parametro di tipo puntatore utilizzato per memorizzare (tramite dereferenziazione) un valore di ritorno all'esterno dello stack frame della funzione.
- Consente a una funzione di restituire valori multipli modificando le variabili nello scope del chiamante.
- **Avvertenza**: L'uso scorretto dei parametri di output rappresenta la causa principale di errori di programmazione nel corso!
- **Ciclo di gestione in 5 fasi**:
  1. Il *Caller* crea lo spazio di memoria per il dato (es. `type var;`).
  2. Il *Caller* passa il puntatore a tale spazio al *Callee* (es. `&var`).
  3. Il *Callee* dichiara un parametro puntatore (es. `type* outparam`).
  4. Il *Callee* scrive il dato dereferenziando il puntatore (es. `*outparam = value;`).
  5. Il *Caller* legge il valore aggiornato accedendo alla variabile originale (`var`).

2.4 Relazione tra Puntatori e Array (Pointers and Arrays)

2.4.1 Equivalenza Sintattica e Indirizzamento

- Un puntatore può puntare ad un qualsiasi elemento di un array.
- L'indicizzazione con parentesi quadre su un puntatore `ptr[i]` è sintatticamente identica all'espressione con aritmetica dei puntatori `*(ptr + i)`.
- Il nome di un array equivale all'indirizzo del suo primo elemento (`&arr[0]`); esso funziona come un puntatore costante e non può essere riassegnato ad un altro indirizzo.

2.4.2 Tracciamento Passo-Passo della Modifica di un Array

- Si consideri il codice sottostante:

```
int a[] = {10, 20, 30, 40, 50};
int* p1 = &a[3]; // punta al 4    elemento (40)
int* p2 = &a[0]; // punta al 1    elemento (10)
int* p3 = a;     // punta al 1    elemento (10)

*p1 = 100;    // a[3] diventa 100
*p2 = 200;    // a[0] diventa 200
```

```
p1[1] = 300; // *(p1 + 1) = a[4] diventa 300
p2[1] = 400; // *(p2 + 1) = a[1] diventa 400
p3[2] = 500; // *(p3 + 2) = a[2] diventa 500
```

- **Evoluzione dei valori dell'array `a[]`**:
  - Iniziale: $\{10, 20, 30, 40, 50\}$.
  - Dopo `*p1 = 100`: $\{10, 20, 30, 100, 50\}$.
  - Dopo `*p2 = 200`: $\{200, 20, 30, 100, 50\}$.
  - Dopo `p1[1] = 300`: $\{200, 20, 30, 100, 300\}$.
  - Dopo `p2[1] = 400`: $\{200, 400, 30, 100, 300\}$.
  - Dopo `p3[2] = 500`: $\{200, 400, 500, 100, 300\}$.
- **Stato finale dell'array `a[]`**: $\{200, 400, 500, 100, 300\}$.

2.4.3 Passaggio di Array come Parametri

- I parametri di tipo array vengono passati alle funzioni **per valore sotto forma di puntatori** al primo elemento.
- La sintassi con le parentesi quadre `[]` nel tipo del parametro è un'agevolazione sintattica a beneficio del lettore.
- Le due dichiarazioni seguenti sono perfettamente equivalenti per il compilatore C:

```
void f(int a[]); // Sintassi convenzionale orientata alla leggibilita
void f(int* a);  // Firma reale interpretata dal compilatore
```

2.5 Puntatori a Funzioni (Function Pointers)

2.5.1 Sintassi e Definizione

- A livello di linguaggio macchina/assembly, il nome di una funzione rappresenta l'indirizzo di memoria in cui risiedono le istruzioni della funzione nel segmento di codice (`.text`).
- È possibile definire puntatori che memorizzano gli indirizzi di memoria di funzioni.
- **Formato generico della dichiarazione**:

```
returnType (*name)(type1, ..., typeN);
```

- Le parentesi tonde attorno a `(*name)` sono strettamente necessarie: senza di esse, il C interpreterebbe la dichiarazione come una funzione che restituisce un puntatore ad un dato (`returnType*`).

2.5.2 Invocazione e Semplificazioni Sintattiche del C

- L'invocazione di una funzione mediante puntatore avviene con la sintassi esplicita:

```
(*name)(arg1, ..., argN);
```

- Il linguaggio C ammette due scorciatoie sintattiche implicite:

  1. È possibile omettere l'operatore `&` nell'assegnamento del nome di una funzione al puntatore.
  2. È possibile omettere l'operatore di dereferenziazione `*` durante la chiamata della funzione (es. `name(arg1, ..., argN)`).

2.5.3 Esempio di Applicazione: La Funzione `map()` (`map.c`)

- La funzione `map()` esegue un'operazione puntata su ciascun elemento di un array:

```
#define LEN 4

int negate(int num) { return -num; }
int square(int num) { return num * num; }

// Applica la funzione puntata 'op' a ogni elemento dell'array
void map(int a[], int len, int (*op)(int n)) {
    for (int i = 0; i < len; i++) {
        a[i] = (*op)(a[i]); // Dereferenziazione del puntatore a funzi
    }
}

int main(int argc, char** argv) {
    int arr[LEN] = {-1, 0, 1, 2};
    int (*op)(int n);   // Dichiarazione puntatore a funzione
    op = square;        // Assegnamento indirizzo funzione
    map(arr, LEN, op);  // Passaggio del puntatore come argomento
    return 0;
}
```

- La sintassi semplificata permette sia la chiamata `a[i] = op(a[i]);` sia il passaggio diretto `map(arr, LEN, square);`.

2.6 Esercizi Supplementari (Extra Exercises)

2.6.1 Esercizio Extra #1: Analisi di Puntatori a Puntatori e Modifica Array

- Si analizzi il comportamento del seguente programma:

```
#include <stdio.h>
#include <stdlib.h>

int foo(int* bar, int** baz) {
    *bar = 5;
    *(bar + 1) = 6;
    *baz = bar + 2;
    return *((*baz) + 1);
}

int main(int argc, char** argv) {
    int arr[4] = {1, 2, 3, 4};
    int* ptr;
    arr[0] = foo(&arr[0], &ptr);
    printf("%d %d %d %d %d\n", arr[0], arr[1], arr[2], arr[3], *ptr);
    return EXIT_SUCCESS;
}
```

- **Tracciamento dettagliato**:
  - `*bar = 5;` modifica `arr[0]` portandolo da 1 a 5.
  - `*(bar + 1) = 6;` modifica `arr[1]` portandolo da 2 a 6.
  - `*baz = bar + 2;` fa sì che `ptr` punti ad `arr[2]` (indirizzo del terzo elemento).
  - `*((*baz) + 1)` valuta l'elemento adiacente ad `arr[2]`, ovvero `arr[3]` (valore 4), e lo restituisce.
  - `arr[0] = foo(...)` sovrascrive `arr[0]` assegnandogli il valore restituito 4.
- **Output stampato a schermo**: `4 6 3 4 3`.

2.6.2 Esercizio Extra #2: Rilevamento Dinamico dell'Endianness

- Richiesta: Scrivere un programma in C che determini e stampi se l'architettura del computer in uso sia Little-Endian o Big-Endian.
- Suggerimento: Fare riferimento all'esempio `pointerarithmetic.c` o allo script `show_bytes.c` del corso CSE 351.

2.6.3 Esercizio Extra #3: Allocazione Dinamica di Array di Puntatori

- Richiesta: Scrivere una funzione che accetti come argomenti un array di interi e la sua lunghezza.
- La funzione deve allocare con `malloc` un array di puntatori a interi (`int*`) della medesima lunghezza.
- Deve inizializzare ciascun elemento dell'array di puntatori allocato affinché punti al corrispondente elemento dell'array di interi passato in ingresso.
- La funzione deve infine restituire il puntatore all'array di puntatori allocato.

2.6.4 Esercizio Extra #4: Invocazione Dinamica tramite Puntatore a Funzione

- Richiesta: Scrivere una funzione che accetti come parametri un puntatore a funzione e un numero intero.
- La funzione deve invocare la funzione puntata passando l'intero fornito come argomento.

3 Puntatori, Parametri di Output e Memoria Dinamica nello Heap

3.1 Informazioni Generali sul Corso ed Esercizi

- **Gestione degli Esercizi (Exercise 1 e 2):**
  - **Exercise 1:** In consegna la sera stessa (02/10) entro le ore 22:00.
  - **Exercise 2:** Rilasciato in data odierna e in scadenza giovedì (05/10) entro le ore 22:00.
  - **Valutazione (totale 8 punti):**
    - **Autograder (6 punti):** Compilazione (1 pt), Correttezza (3 pt), Linter (1 pt), Valgrind (1 pt). I punteggi dell'Autograder sono visibili subito dopo la scadenza; le soluzioni di esempio vengono rilasciate lo stesso giorno della scadenza di consegna in ritardo.
    - **Valutazione Manuale (2 punti):** Stile del codice e aderenza alle linee guida.
  - **Indicazioni di Stile:** Seguire rigorosamente la *Style Guide* del corso e la *Google C++ Style Guide*. Documentare e motivare eventuali scelte progettuali.
- **Homework (hw0 e hw1):**

- **hw0:** In scadenza martedì entro le 22:00:00.
- **Procedura Git per hw0:** Eseguire `git add`, `commit`, `push`, creare il tag `hw0-final` e inviare il tag (`git push --tags`). Successivamente, clonare il repository in una cartella differente, eseguire `git checkout hw0-final` e verificare la correttezza del contenuto.
- **hw1:** In rilascio il giorno successivo. È severamente vietato modificare i file di interfaccia (`.h`), ma è necessario leggerli attentamente durante l'implementazione. Tutti i bug riscontrati vanno tracciati all'interno di `bugjournal.md`.

- **Documentazione e Risorse:**
  - **Fonti Ufficiali:** Pagine `man`, libri di testo, siti di riferimento (`cplusplus.org`, `man7.org`, `gcc.gnu.org`).
  - **Fonti Informali ("Folklore"):** Ricerche su Google, Stack Overflow, colleghi di laboratorio, modelli AI (ChatGPT). Viene sottolineata l'importanza di comprendere i punti di forza e di debolezza relativi tra documentazione ufficiale e risorse informali.

3.2 Parametri di Output (Output Parameters)

Un **parametro di output** è un parametro di tipo puntatore utilizzato per memorizzare (tramite dereferenziazione) il risultato di una funzione all'esterno dello stack frame della funzione stessa.

- **Scopo e Utilità:** Consente di modificare variabili situate nello scope del chiamante (*Caller*) ed è fondamentale quando una funzione deve restituire più valori di ritorno.
- **Avviso:** L'uso scorretto dei parametri di output costituisce la principale causa di errori nei progetti di questo corso.
- **Procedura di Invocazione in 5 Passaggi:**
  1. Il *Caller* alloca lo spazio di memoria per il dato (es. `type var;`).
  2. Il *Caller* passa l'indirizzo di memoria al *Callee* (es. `&var`).
  3. Il *Callee* accetta il puntatore come parametro di output (es. `type* outparam`).
  4. Il *Callee* assegna il valore dereferenziando il puntatore (es. `*outparam = value;`).
  5. Il *Caller* accede al risultato modificato tramite la propria variabile (`var`).

3.2.1 Analisi dei Casi di Invocazione di `GenerateString`

Data la funzione:

```
void GenerateString(char** output) {
    *output = "Hello there\n";
}
```

Vengono analizzate le seguenti modalità di chiamata:

- **Opzione A (*Errata*):**

  ```
  char** result;
  GenerateString(result);
  printf("%s", *result);
  ```

  Errore: `result` non è inizializzato. Dereferenziare dati indefiniti causa comportamenti imprevedibili o Segmentation Fault.

- **Opzione B (*Funzionante ma sconsigliata*):**

  ```
  char* result[1] = {NULL};
  GenerateString(result);
  printf("%s", result[0]);
  ```

  Alloca un array di puntatori. Funziona poiché il nome dell'array decade a puntatore al primo elemento (`char**`), ma sprecando memoria inutilmente.

- **Opzione C (*Funzionante ma poco chiara*):**

  ```
  char* str;
  char** result = &str;
  GenerateString(result);
  printf("%s", str);
  ```

- **Opzione D (*Uso Preferito – Preferred Usage*):**

  ```
  char* result;
  GenerateString(&result);
  printf("%s", result);
  ```

  Rappresenta l'approccio ideale: evita errori di inizializzazione, minimizza l'uso della memoria ed esprime l'intenzione del codice in modo limpido.

3.3 Puntatori a Funzione (Function Pointers)

In linguaggio assembly, il nome di una funzione rappresenta un indirizzo di memoria (*label address* o valore del Program Counter). Di conseguenza, in C è possibile definire puntatori che memorizzano l'indirizzo di codice di una funzione.

- **Sintassi di Dichiarazione Generica:**

  `returnType (*name)(type1, ..., typeN);`

  Nota: Le parentesi attorno a `(*name)` sono obbligatorie; altrimenti la dichiarazione verrebbe interpretata come il prototipo di una funzione che restituisce un puntatore (`returnType* name(...)`).

- **Invocazione:**

  `(*name)(arg1, ..., argN);`

- **Sintassi Semplificata in C (Shorthand):**
  - Il compilatore C consente di omettere l'operatore `&` durante l'assegnazione del nome della funzione (il nome della funzione valuta automaticamente all'indirizzo, analogamente agli array).
  - È possibile omettere l'operatore di dereferenziazione `*` nella chiamata: `name(arg1, ..., argN)` equivale a `(*name)(arg1, ..., argN)`.

- **Esempio Pratico (`Map`):**

```
#define LEN 4

int Negate(int num) { return -num; }
int Square(int num) { return num * num; }

void Map(int a[], int len, int (*op)(int n)) {
    for (int i = 0; i < len; i++) {
        a[i] = op(a[i]); // Dereferenziazione implicita
    }
}

int main(int argc, char** argv) {
    int arr[LEN] = {-1, 0, 1, 2};
    Map(arr, LEN, Square); // Ometto & e *
    return 0;
}
```

3.4 Allocazione Dinamica della Memoria e Lo Heap

L'allocazione statica (variabili globali) ed automatica (sullo *Stack*) risulta insufficiente nelle seguenti tre situazioni:

Necessità di memoria con durata di vita persitente oltre l'esecuzione della singola funzione, ma non estesa all'intera durata del programma.

Necessità di allocare strutture dati la cui dimensione supera la capacità dello Stack.

Dimensione dei dati non nota a priori in fase di compilazione (es. lettura di un file di dimensione variabile a runtime).

3.4.1 Il Ruolo di `NULL`

- Nelle architetture C su Linux, `NULL` corrisponde all'indirizzo `0x0`.
- La dereferenziazione di `NULL` garantisce un *Segmentation Fault* (segfault).
- Viene utilizzato come indicatore di puntatore non inizializzato, non in uso o di fallimento dell'allocazione. Causare un segfault immediato è nettamente preferibile rispetto alla corruzione silenziosa della memoria.

3.5 Gestione della Memoria con `malloc()` e `free()`

3.5.1 La Funzione `malloc()`

`malloc` alloca un blocco contiguo di memoria non inizializzata nello Heap di dimensione pari ad almeno i byte richiesti.

```
var = (type*) malloc(size_in_bytes);
```

- **Valore di Ritorno:** Restituisce un puntatore al primo byte della memoria allocata; restituisce `NULL` se l'allocazione fallisce.
- **Regole di Stile e Buone Pratiche:**
  1. Utilizzare sempre l'operatore `sizeof` nell'argomento (es. `10 * sizeof(float)`).
  2. Effettuare il cast esplicito del tipo del valore restituito.
  3. **Verificare sempre** che il valore restituito non sia `NULL` prima di accedere alla memoria.
- **Funzioni Correlate:** `calloc()` (alloca e azzera la memoria) e `realloc()` (ridimensiona un blocco allocato precedentemente).

3.5.2 La Funzione `free()`

Dealloca il blocco di memoria nello Heap precedentemente allocato.

```
free(pointer);
```

- Il puntatore passato a `free` deve obbligatoriamente puntare al primo byte restituito da una precedente chiamata a `malloc` o `calloc`.
- La memoria liberata torna a disposizione per future allocazioni dello Heap.
- Eseguire `free(NULL)` non ha alcun effetto ed è un'operazione sicura.
- **Puntatori Pendenti (*Dangling Pointers*):** La chiamata a `free()` **non modifica** il valore dell'indirizzo contenuto nella variabile puntatore.
- **Programmazione Difensiva:** È raccomandato impostare il puntatore a `NULL` subito dopo la `free()` (es. `ptr = NULL;`) in contesti di debugging o codice non critico per le prestazioni.

3.6 Esempio di Esecuzione e Gestione Stack/Heap (`arraycopy.c`)

Viene esaminato il comportamento della memoria durante la copia dinamica di un array:

```
#include <stdlib.h>

int* Copy(int a[], int size) {
    int i, *a2;
    a2 = (int*) malloc(size * sizeof(int));
    if (a2 == NULL)
        return NULL;
    for (i = 0; i < size; i++)
        a2[i] = a[i];
    return a2;
}

int main(int argc, char** argv) {
    int nums[4] = {1, 2, 3, 4};
    int* nums_copy = Copy(nums, 4);
    // ... utilizzo dell'array ...
    free(nums_copy);
    return EXIT_SUCCESS;
}
```

- **Esecuzione:** Durante l'invocazione di `Copy`, nello Stack viene creato lo frame per `Copy`. `malloc` alloca 16 byte (4 `int`) nello Heap.

- **Ritorno:** Alla fine di `Copy`, il suo frame di Stack viene distrutto, ma la memoria nello Heap rimane integra e il suo indirizzo viene restituito a `nums_copy` nel frame di `main`.
- **Deallocazione:** `free(nums`$_{c}$*opy*)*deallocailblocconelloHeap.Successivamente*, `nums_copy`*divent*

3.7 Corruzione della Memoria e Memory Leak

3.7.1 Errori Comuni di Memoria (`memcorrupt.c`)

Nel codice d'esempio vengono analizzati i seguenti errori fatali:

1. **Scrittura Out-of-Bounds:** `a[2] = 5;` su un array `int a[2];` alloca nello Stack (sovrascrittura dello Stack).
2. **Mancata verifica di `NULL`:** `b[0] += 2;` senza aver controllato se `malloc` ha restituito `NULL`.
3. **Puntatore fuori blocco:** `c = b + 3;` calcola un indirizzo oltre la memoria allocata.
4. **Free di indirizzo di Stack:** `free(&(a[0]));` tentativo illegale di liberare memoria dello Stack tramite `free()`, provocando il crash del programma.
5. **Double Free:** `free(b); free(b);` liberazione duplicata dello stesso indirizzo di memoria.
6. **Use-After-Free:** `b[0] = 5;` scrittura tramite un puntatore la cui memoria è già stata deallocata.

3.7.2 Perdite di Memoria (Memory Leak)

Si verifica un **memory leak** quando il programma omette di deallocare memoria dello Heap che non è più utilizzata (ad esempio dimenticando di chiamare `free()` o sovrascrivendo l'unico puntatore al blocco).

- **Complessità della Gestione:** Il passaggio di puntatori tra varie funzioni rende spesso complessa l'assegnazione della responsabilità di deallocazione della memoria.
- **Conseguenze:** L'impronta di memoria virtuale (*virtual memory footprint*) del processo cresce continuamente. Sebbene nei programmi brevi l'intero spazio di memoria venga liberato dal sistema operativo al termine dell'esecuzione, nei programmi a lunga esecuzione (*long-lived programs*) i memory leak provocano un grave degrado delle prestazioni ed il successivo esaurimento della memoria.

3.8 Esercizi Extra Proposti

1. **Esercizio Extra #1:** Scrivere una funzione che accetti come argomenti un puntatore a funzione e un intero, e invochi la funzione puntata passando l'intero come parametro.

2. **Esercizio Extra #2:** Scrivere una funzione che accetti una stringa in ingresso e restituisca:

   - La prima parola (separata da spazi) allocata dinamicamente come nuova stringa nello Heap.
   - La dimensione/lunghezza di tale parola.

3. **Esercizio Extra #3:** Scrivere una funzione che accetti come argomenti un array di `int` e la sua lunghezza, allochi dinamicamente con `malloc` un array di puntatori a `int` della medesima lunghezza, inizializzi ciascun elemento affinché punti al corrispondente elemento dell'array originale, e restituisca il puntatore all'array di puntatori così creato.

4 Puntatori a Puntatori, Stringhe, Strutture e Strumenti di Sviluppo

4.1 Parte 1: Puntatori a Puntatori (Double Pointers)

4.1.1 Anatomia e Dereferenziazione

Un puntatore a puntatore è una variabile che memorizza l'indirizzo di memoria di un altro puntatore. Ogni operatore di dereferenziazione (`*`) segue un livello di indirezione:

- `int x = 42;` → allocato all'indirizzo `0x1000`.
- `int *p = &x;` → allocato all'indirizzo `0x2000`, contiene il valore `0x1000`.
- `int **pp = &p;` → allocato all'indirizzo `0x3000`, contiene il valore `0x2000`.

Valori e tipi delle espressioni:

- `pp`: tipo `int**`, valore `0x2000` (indirizzo di `p`).
- `*pp`: tipo `int*`, valore `0x1000` (valore di `p`, ovvero `&x`).
- `**pp`: tipo `int`, valore `42` (valore contenuto in `x`).

4.1.2 Uso 1: Parametri di Output (Out-Parameters)

In C tutti gli argomenti vengono passati per valore, inclusi i puntatori.

- **Il bug del passaggio per valore:** Se si tenta di allocare memoria all'interno di una funzione passandole un puntatore semplice:

```
void alloc_array(int *a, int n) {
    a = malloc(n * sizeof(int));
}
```

  La variabile `a` è una copia locale di `arr`. L'assegnazione della memoria allocata non modifica `arr` nel `main` (che rimane `NULL`), causando un crash all'accesso e un *memory leak* al termine della funzione.

- **La soluzione con doppi puntatori:** Per modificare una variabile di tipo `int`, occorre passare un `int*`; per modificare un `int*`, occorre passare un `int**`:

```
void alloc_array(int **a, int n) {
    *a = malloc(n * sizeof(int));
}
// Chiamata nel main:
int *arr = NULL;
alloc_array(&arr, 10); // *a all'interno della funzione modifica direttamen
```

- **Pattern alternativi e librerie standard:** Se possibile, la soluzione più semplice consiste nel restituire direttamente il puntatore tramite `return`. Tuttavia, si usa un parametro di output (`int **out`) quando il valore di ritorno della funzione è già riservato al codice di stato/errore. Esempi nelle librerie POSIX/C standard:
  - `getline(char **lineptr, size`$_t$`*n,` $FILE*f$`)` : *accettaundoppiopuntatoreperchépuòriall*
  - `strtok: traccia la posizione in cui si è interrotta l'analisi sintattica.`
  - `posix`$_m$$emalign(void**memptr, size_talign, size_tsize)$: *restituisceilcodiced'errorecomeval*

4.1.3 Uso 2: Array di Puntatori e Matrici

Un altro utilizzo fondamentale dei doppi puntatori è la gestione di vettori di puntatori:

- **Gestione degli argomenti CLI (`argv`):** Nel `main(int argc, char **argv)`, `argv` è un array di stringhe (`char*`). Ad esempio, eseguendo `./prog hello 42` (`argc == 3`):
  - `argv[0]` → `"./prog"`
  - `argv[1]` → `"hello"` (con `argv[1][0] == 'h'`)
  - `*argv[2]` → `'4'`
  - `argv[3]` → `NULL` (elemento terminatore)
- **Matrice come array di puntatori a righe (`int **m`):**

```
int **m = malloc(rows * sizeof(int*));
for (int i = 0; i < rows; i++) {
    m[i] = malloc(cols * sizeof(int));
}
m[1][2] = 7;
// Liberazione (prima le singole righe, poi l'array principale):
for (int i = 0; i < rows; i++) free(m[i]);
free(m);
```

  Costo: Richiede `rows + 1` chiamate a `malloc` e `free`.
- **Matrice in un singolo blocco contiguo (Preferibile per codice numerico):**

```
int *m = malloc(rows * cols * sizeof(int));
m[i * cols + j] = 7; // Indicizzazione manuale
free(m);
```

- **Confronto dei due approcci per matrici:**
  - *Chiamate malloc/free:* `rows + 1` per `int**` contro `1` per il blocco contiguo.
  - *Layout di memoria:* Sparso nell'heap per `int**` contro blocco contiguo *cache-friendly* per il blocco unico.
  - *Interoperabilità (MPI, BLAS, NumPy):* `int**` richiede il passaggio riga per riga; il blocco unico viene passato direttamente come singolo buffer.
  - *Righe di lunghezza differente (Jagged Arrays):* Supportato da `int**`, non supportato dal blocco unico.

4.2 Parte 2: Stringhe

4.2.1 Natura delle Stringhe e Byte Nullo

In C non esiste un tipo di dato nativo per le stringhe. Una stringa è un array di caratteri (`char[]`) con la convenzione formale che il testo termina con il carattere nullo `'\0'`.

- `char s[] = "hi!";` equivale a `char t[] = {'h', 'i', '!', '\0'};`.
- `strlen(s)` restituisce `3` (numero di caratteri prima di `'\0'`).
- `sizeof(s)` restituisce `4` (dimensione totale dell'array in byte, incluso `'\0'`).
- Omettere `'\0'` porta le funzioni di libreria a leggere indefinitamente la memoria finché non incontrano accidentalmente un byte zero (comportamento indefinito).

4.2.2 Array vs Puntatori a Literals

- **Array (`char a[] = "hello";`):** Crea una copia modificabile allocata nello stack. `a[0] = 'H'` è valido; `sizeof(a)` vale `6`; reassegnare l'array (`a = "bye"`) genera un errore di compilazione.
- **Puntatore (`char *p = "hello";`):** Punta a una stringa letterale situata in memoria di sola lettura (Read-Only). Scrivere `p[0] = 'H'` causa un crash a runtime (undefined behavior); `sizeof(p)` restituisce la dimensione del puntatore (`8` byte su architetture a 64 bit); riorientare il puntatore (`p = "bye"`) è consentito.
- **Buona pratica:** Usare sempre `const char *p = "hello";` così il compilatore blocca eventuali tentativi di scrittura già in fase di compilazione.

4.2.3 Corrispondenza tra Operazioni Python e Funzioni C

- `len(s)` → `strlen(s)` (*Attenzione:* ha complessità $\mathcal{O}(n)$, scorre la memoria fino a `'\0'`).
- `s == t` → `strcmp(s, t) == 0` (*Attenzione:* in C `s == t` confronta gli indirizzi di memoria, non il contenuto).
- `t = s[:]` → `strcpy(t, s)` (*Attenzione:* la destinazione `t` deve avere spazio sufficiente allocato).
- `s + t` → `strcat(s, t)` (*Attenzione:* `s` deve avere spazio libero residuo per contenere entrambe le stringhe più `'\0'`).
- `f"{n} items"` → `snprintf(buf, sizeof buf, "%d items", n)` (tronca in modo sicuro ed evita buffer overflow).

- `c in s` → `strchr(s, c) != NULL` (restituisce un puntatore alla prima occorrenza o `NULL`).
- `int(s)` → `strtol(s, &end, 10)` (permette di verificare il puntatore `end` per rilevare errori di conversione).

4.2.4 Confronti di Stringhe ed Errori Frequenti

L'utilizzo dell'operatore `==` con le stringhe in C confronta gli indirizzi di memoria.

```
char answer[16];
fgets(answer, sizeof(answer), stdin);
answer[strcspn(answer, "\n")] = '\0'; // Rimozione del carattere new-line

if (answer == "yes") { ... }  // MAI VERO: confronta due indirizzi differenti
if (strcmp(answer, "yes") == 0) { ... } // CORRETTO: confronta i singoli caratteri
```

Con il flag `-Wall`, GCC segnala l'avviso: `warning: comparison with string literal results in unspecified behavior [-Waddress]`.

4.2.5 Allocazione e Restituzione di Nuove Stringhe

Per restituire una nuova stringa da una funzione, occorre allocare spazio sufficiente per la lunghezza del testo più 1 byte per il terminatore nullo:

```
char *my_strdup(const char *s) {
    size_t n = strlen(s) + 1; // +1 per '\0'
    char *copy = malloc(n);
    if (copy == NULL) return NULL;
    memcpy(copy, s, n);
    return copy; // Il chiamante diventa proprietario della memoria
}
```

- **Gestione della proprietà:** La funzione che restituisce memoria heap deve documentarlo chiaramente e il chiamante ha la responsabilità di chiamare `free()`.
- **Errore fatale:** Non restituire mai un array locale allocato sullo stack (es. `char buf[64]; return buf;`), in quanto il buffer viene distrutto al termine della funzione rendendo il puntatore pendente (*dangling pointer*).

4.3 Parte 3: Strutture (`struct`)

4.3.1 Dal Dataclass di Python alla `struct` in C

Una `struct` in C è assimilabile a una classe priva di metodi e di ereditarietà, la cui disposizione in memoria (*layout*) è totalmente prevedibile byte per byte.

- **Dichiarazione e istanziazione:**

```
typedef struct {
    double x;
    double y;
} Point;

Point p = {.x = 1.0, .y = 2.0};
p.x += 1;
```

- I campi sono fissati in fase di compilazione (non è possibile aggiungere attributi dinamicamente).
- I "metodi" sono normali funzioni esterne che accettano la struttura come parametro (es. `point`$_n orm(\&p)$).

4.3.2 Assegnazione, Copia per Valore e Operatore ->

- **Copia per valore:** A differenza di Python dove `q = p` assegna un riferimento allo stesso oggetto, in C l'assegnazione `Point q = p;` esegue una copia bit a bit (*shallow copy*) indipendente. Modificare `q.x` non altera `p.x`.
- **Passaggio a funzione:** Le strutture vengono passate per valore. Per modificarle o evitare costi di copia per strutture di grandi dimensioni, occorre passare un puntatore.
- **Accesso tramite puntatore (->):** L'operatore `p->x` è una notazione sintattica equivalente a `(*p).x`.

```
void point_scale(Point *p, double k) {
    p->x *= k;
    p->y *= k;
}
```

4.3.3 Strutture Autoreferenziali: Liste Concatenate

Una struttura può contenere un puntatore a un'altra istanza dello stesso tipo:

```
typedef struct node {
    int value;
    struct node *next;
} Node;
```

```
// Inserimento in testa:
Node *push(Node *head, int v) {
    Node *n = malloc(sizeof *n);
    n->value = v;
    n->next = head;
    return n; // Nuova testa
}

// Scorrimento:
for (Node *p = list; p; p = p->next) {
    printf("%d ", p->value);
}

// Deallocazione corretta della lista:
while (list) {
    Node *next = list->next;
    free(list);
    list = next;
}
```

4.3.4 Allineamento in Memoria e Padding

L'ordine dei campi all'interno di una `struct` influisce sulla sua dimensione complessiva a causa del riempimento di allineamento (*padding*) inserito dal compilatore per motivi di efficienza dell'architettura hardware (es. x86-64):

```
struct a { char c; double d; char e; }; // sizeof: 24 byte su x86-64
struct b { double d; char c; char e; }; // sizeof: 16 byte
```

- **Layout di `struct a`:** `c` (1 byte) + 7 byte padding + `d` (8 byte) + `e` (1 byte) + 7 byte padding = 24 byte.
- **Layout di `struct b`:** `d` (8 byte) + `c` (1 byte) + `e` (1 byte) + 6 byte padding = 16 byte.

4.4 Parte 4: Strumenti di Sviluppo e Debugging

Mentre Python solleva eccezioni esplicite a runtime, il linguaggio C corrompe silenziosamente la memoria. Gli strumenti di sviluppo permettono di rilevare tempestivamente questi errori.

4.4.1 Flag del Compilatore GCC

I warning del compilatore fungono da controllo sui tipi:

```
gcc -std=c17 -Wall -Wextra -g -O0 -o prog prog.c
```

- `-Wall -Wextra`: Abilitano i warning fondamentali per identificare i bug principali (non attivi di default).
- `-Werror`: Trasforma ogni warning in un errore bloccante.
- `-g`: Mantiene le informazioni di debug (numeri di riga e nomi delle variabili) per GDB e i sanitizers.
- `-O0`: Disabilita le ottimizzazioni durante la fase di debugging (utilizzare `-O3` per i benchmark finali).
- `-std=c17`: Fissa la versione del linguaggio standard rendendo il comportamento portabile tra compilatori diversi.

4.4.2 Sanitizer (AddressSanitizer e UBSan)

I sanitizer arrestano immediatamente l'esecuzione dell'applicazione mostrando la riga esatta dell'errore:

```
gcc -g -fsanitize=address,undefined bug.c -o bug
```

- **AddressSanitizer (ASan):** Intercetta accessi fuori dai limiti dell'array (*heap-buffer-overflow*), *use-after-free*, *double free* e *memory leak*.
- **UndefinedBehaviorSanitizer (UBSan):** Rileva *overflow* su interi con segno e operazioni di *shift* non valide.
- Introducono un rallentamento contenuto ($\approx 2\times$): raccomandati in tutti i test di esecuzione.

4.4.3 Il Debugger GDB

Comandi principali per l'analisi del codice:

- `gdb ./prog`: Carica il programma compilato con il flag `-g`.
- `break main` o `b list.c:42`: Imposta un punto di interruzione (*breakpoint*).
- `run arg1 arg2`: Avvia l'esecuzione passando i parametri specificati.
- `next`: Esegue la riga successiva scavalcando le chiamate a funzione.

- `step`: Entra all'interno della funzione chiamata.
- `print *node` / `p arr[0]@5`: Ispeziona un valore o stampa i primi 5 elementi di un array.
- `backtrace`: Mostra la catena di chiamate (*call stack*) che ha generato un crash (*segmentation fault*).
- `watch p->next`: Interrompe l'esecuzione ogni volta che il valore specificato cambia.

4.4.4 Valgrind

Analizzatore dinamico che non richiede la ricompilazione del codice:

```
valgrind --leak-check=full ./prog
```

- **Confronto con AddressSanitizer:**
  - *Ricompilazione necessaria:* Sì per ASan (`-fsanitize`), No per Valgrind.
  - *Impatto sulle prestazioni:* ASan rallenta l'esecuzione di circa $2\times$, Valgrind di circa $20 \times -50\times$.

4.4.5 Organizzazione Multi-file e Makefile

Progettazione di moduli software riutilizzabili:

- **File d'intestazione (`point.h`):** Contiene le dichiarazioni di tipi e i prototipi delle funzioni, protetti da un *include guard*:

```
#ifndef POINT_H
#define POINT_H
typedef struct { double x, y; } Point;
double point_norm(const Point *p);
#endif
```

- **File di sorgente (`point.c`):** Contiene le definizioni effettive della logica:

```
#include <math.h>
#include "point.h"
double point_norm(const Point *p) {
    return hypot(p->x, p->y);
}
```

- **File di automazione (`Makefile`):** I rientri dei comandi devono utilizzare tassativamente il carattere TAB reale.

```
CC = gcc
CFLAGS = -std=c17 -Wall -Wextra -g
LDLIBS = -lm

prog: main.o point.o
    $(CC) $^ -o $@ $(LDLIBS)

main.o point.o: point.h

clean:
    rm -f prog *.o
```

4.5 Regole d'Oro Conclusive (Take-aways)

1. **Prevedere sempre `len + 1`:** Ogni stringa in C è un array di `char` seguito dal carattere `'\0'`. Ogni buffer di destinazione deve riservare lo spazio per tale byte.
2. **Le strutture vengono copiate:** L'assegnazione e il passaggio di parametri duplicano tutti i byte. Passare un puntatore per condividere o modificare la struttura originale.
3. **Ogni `malloc` ha un proprietario esplicito:** Definire e documentare chi deve liberare la memoria; liberare prima i componenti interni e successivamente il contenitore principale.
4. **Lasciar "gridare" gli strumenti di sviluppo:** Usare sempre i flag `-Wall -Wextra -g`, i sanitizers (`-fsanitize=address,undefined`), GDB e Valgrind per identificare subito le corruzioni di memoria.

5 Introduzione al Corso di Multicore Programming

5.1 Motivazioni Fisiche e Architetturali

L'esplosione dell'Intelligenza Artificiale (guidata dalla disponibilità di dati, modelli complessi e potenza di calcolo) e la necessità di risolvere problemi ad alta complessità scientifica (LLM, genomica, modellazione climatica, protein folding, fisica del CERN) hanno spinto la domanda di calcolo a livelli senza precedenti.

- **Fine dello scaling di frequenza:** Dal 1986 al 2003 le prestazioni dei microprocessori crescevano in media del 50% all'anno (60× in 10 anni). Dal 2003 questo incremento è fortemente rallentato (soltanto +4% all'anno tra il 2015 e il 2017).
- **Power Wall e limiti fisici:** Transistor più piccoli consentono frequenze maggiori, ma generano un consumo energetico ed un calore eccessivi, rendendo i processori monolitici inaffidabili.
- **Transizione al Multicore e GPU:** Per superare i limiti termici ed energetici, l'industria ha virato verso l'integrazione di più nuclei di calcolo (core) su un singolo circuito integrato e l'uso di acceleratori (GPU da decine di miliardi di transistor).

5.2 Programmazione Parallela e Prestazioni

I programmi seriali tradizionali non traggono beneficio automatico dalle architetture multicore.

- **Limiti della parallelizzazione automatica:** Convertire automaticamente codice seriale in parallelo tramite compilatori è un processo complesso e spesso inefficiente. Spesso la soluzione migliore richiede un cambio di paradigma e la riprogettazione completa dell'algoritmo.
- **Linguaggio di riferimento:** Il corso adotta il linguaggio **C** per garantire il controllo diretto della memoria e massimizzare le prestazioni hardware.
- **Evoluzione dello Speedup:** L'impiego di linguaggi a basso livello, costrutti paralleli e ottimizzazioni architetturali consente di ottenere incrementi prestazionali di diversi ordini di grandezza:

Versione	Implementazione	Speedup Assoluto	Speedup Relativo
1	Python	1	1,0
2	Java	11	10,8
3	C	47	4,4
4	Parallel Loop	366	7,8
5	Divide and Conquer	6.727	18,4
6	Vettorizzazione	23.224	3,5
7	AVX Intrinsics	62.806	2,7

Tabella 2: Confronto prestazionale tra diverse strategie di implementazione.

6 Programmazione Parallela e Multicore: Principi, Algoritmi e Architetture

6.1 Motivazioni e Limiti della Parallelizzazione Automatica

- **Transizione al Parallelismo**: Le leggi della fisica e i limiti termici hanno spinto l'informatica verso le tecnologie multicore, consentendo ad oggi applicazioni in grado di utilizzare oltre 200.000 GPU simultaneamente.
- **Inefficienza della Conversione Automatica**: I compilatori possono riconoscere e convertire automaticamente alcuni costrutti di codice sequenziale in costrutti paralleli, ma questo processo risulta frequentemente inefficiente.
- **Riprogettazione Algoritmica**: I programmi seriali non traggono beneficio diretto dai core multipli. Per ottenere prestazioni elevate non è sufficiente parallelizzare il codice seriale esistente, ma occorre spesso fare un passo indietro e progettare ex novo algoritmi espressamente paralleli.

6.2 Studio di Caso: Algoritmi per il Calcolo della Somma Globale

- **Problema Seriale**: Calcolo di n valori e della loro sommatoria tramite un ciclo sequenziale:

```
sum = 0;
for (i = 0; i < n; i++) {
    x = compute_next_value(...);
    sum += x;
}
```

- **Soluzione Parallela Iniziale (Partizionamento Locale)**:
  - Dati p core con $p \ll n$, ogni core calcola autonomamente la somma di n/p valori eseguendo il codice in modo indipendente con variabili private (`my_sum`, `my_first_i`, `my_last_i`, `my_i`, `my_x`).
  - Esempio con $p = 8$ core e $n = 24$ valori (1, 4, 3, 9, 2, 8, 5, 1, 1, 6, 2, 7, 2, 5, 0, 4, 1, 8, 6, 5, 1, 2, 3, 9):

Core	0	1	2	3	4	5	6	7
`my_sum`	8	19	7	15	7	13	12	14

- **Somma Globale tramite Master Core**:
  - Una volta completata la fase locale, un core "master" (Core 0) riceve sequenzialmente i valori parziali da tutti gli altri $p - 1$ core e li somma.

- Gli altri $p-1$ core inviano il proprio `my_sum` al master e terminano.
- *Collo di bottiglia*: Il master esegue tutto il lavoro di riduzione mentre gli altri $p-1$ core rimangono inattivi, richiedendo $p-1$ operazioni di ricezione e addizione seriali (es. 999 operazioni per $p = 1000$).

- **Soluzione Ottimizzata a Struttura ad Albero (Tree-Structured Reduction)**:
  - I core vengono accoppiati a coppie per sommare i risultati in parallelo (es. Core 0+1, 2+3, 4+5, 6+7).
  - Il processo si ripete ricorsivamente solo sui core con indice pari (0+2, 4+6), e infine tra i superstiti (0+4).
  - La complessità temporale si riduce da $p-1$ a $\log_2(p)$ passaggi di ricezione e addizione (es. con $p = 1000$ si passa da 999 a sole 10 operazioni).

- **Ostacoli all'Ottimizzazione del Compilatore**: Se la funzione `compute_next_value(...)` possiede uno stato interno mutabile aggiornato a ogni iterazione, le iterazioni diventano interdipendenti e la parallelizzazione automatica fallisce.

6.3 Tassonomia del Parallelismo e Requisiti di Coordinamento

- **Data Parallelism vs Task Parallelism**:
  - *Data Parallelism (Parallelismo di Dati)*: Partiziona i dati tra i core; ciascun core esegue operazioni simili sulla propria porzione (analogo al parallelismo spaziale nei circuiti).
  - *Task Parallelism (Parallelismo di Task)*: Partiziona compiti distinti tra i core (analogo al parallelismo temporale/pipelining nei circuiti).
  - *Esempio di analogia (Correzione esami - 300 esami, 15 domande, 3 assistenti)*:
    - Data Parallelism: Ogni assistente corregge 100 esami completi.
    - Task Parallelism: L'Assistente 1 corregge le domande 1-5 di tutti gli esami, l'Assistente 2 le domande 6-10, l'Assistente 3 le domande 11-15.
  - *Applicazione all'esempio della somma*: Il calcolo delle somme parziali locali è *Data Parallelism*, mentre la gestione dell'invio e della ricezione della riduzione globale rappresenta *Task Parallelism*.

- **Fattori di Coordinamento tra Core**:
  - *Comunicazione*: Scambio esplicito o implicito di dati tra i core (es. invio di somme parziali).
  - *Bilanciamento del carico (Load Balancing)*: Distribuzione equa del lavoro per prevenire che un core sia sovraccaricato e gli altri $p-1$ rimangano inattivi in attesa del più lento, evitando uno spreco di risorse ed energia.

– *Sincronizzazione*: Coordinamento dei ritmi di esecuzione per evitare che alcuni core procedano troppo velocemente rispetto agli altri.

6.4 Architetture di Memoria, Modelli di Esecuzione e API

- **Organizzazione della Memoria**:
  - *Shared-Memory (Memoria Condivisa)*: Tutti i core condividono l'accesso alla memoria centrale e si coordinano esaminando e aggiornando locazioni condivise.
  - *Distributed-Memory (Memoria Distribuita)*: Ogni core/nodo ha una memoria privata esclusiva e comunica inviando messaggi trasmessi via rete.
- **Modelli d'Istruzione**:
  - *MIMD (Multiple-Instruction Multiple-Data)*: Ogni core possiede una propria unità di controllo ed esegue istruzioni indipendenti su dati differenti.
  - *SIMD (Single-Instruction Multiple-Data, es. GPU)*: I core condividono l'unità di controllo e devono eseguire la medesima istruzione nello stesso istante, altrimenti rimangono inattivi (*idle*).
- **Matrice delle API C Esplicite**:

	Shared Memory	**Distributed Memory**
SIMD	CUDA	–
MIMD	Pthreads / OpenMP / CUDA	MPI

  - Le librerie ad alto livello semplificano lo sviluppo ma sacrificano le prestazioni in favore della facilità d'uso.
- **Distinzioni Concettuali**:
  - *Concorrenza*: Più task possono essere in esecuzione nello stesso intervallo temporale (anche serialmente su un solo core tramite multitasking).
  - *Parallelismo*: Più task cooperano strettamente per risolvere un problema comune (sistemi fortemente accoppiati via memoria condivisa o rete ad alta velocità).
  - *Sistemi Distribuiti*: Programmi distinti cooperano per risolvere un problema (debolmente accoppiati, es. tramite Internet).

6.5 Hardware, Architettura di von Neumann e Costi di Latenza

- **Necessità di Conoscenza Hardware**: Ignorare l'hardware sottostante impedisce la scrittura di codice altamente ottimizzato ed efficiente.
- **Architettura di von Neumann**:
  - *Componenti*: CPU (Unità di Controllo e Datapath), Memoria Principale (collezione di celle indirizzabili contenenti dati/istruzioni), Registri veloci e Interconnessione (bus o rete).
  - *Program Counter (PC)*: Registro dell'unità di controllo contenente l'indirizzo della prossima istruzione da eseguire.
  - *von Neumann Bottleneck*: Separazione fisica tra CPU e memoria che limita la velocità di esecuzione alla frequenza di trasferimento dell'interconnessione.
- **Ordini di Grandezza delle Latenze di Memoria e Sistema**:

Operazione	**Latenza**
Riferimento Cache L1	0,5 ns
Errore di predizione del ramo (Branch mispredict)	5 ns
Riferimento Cache L2	7 ns
Lock/Unlock di un Mutex	25 ns
Riferimento alla Memoria Principale (RAM)	100 ns
Compressione 1 KB con Zippy	3 μs
Invio 1 KB su rete 1 Gbps	10 μs
Lettura casuale SSD (1 Gb/s)	150 μs
Lettura sequenziale 1 MB da Memoria RAM	250 μs
Round trip nello stesso Datacenter	500 μs
Lettura sequenziale 1 MB da SSD	1 ms
Seek su Disco Rigido Meccanico	10 ms
Lettura sequenziale 1 MB da Disco Meccanico	20 ms
Roundtrip di pacchetto CA - Olanda	150 ms

- **Analogia Umana del Tempo di Calcolo (Scala dove 1 Ciclo CPU = 1 Secondo)**:
  - Accesso Cache L1: 3 secondi.
  - Accesso Memoria Principale: 6 minuti.
  - Lettura 1 MB sequenziale da SSD: 200 giorni.
  - Seek su disco meccanico: 4 mesi.
  - Pacchetto di rete SF - NYC: 7 anni.

6.6 Conclusioni e Linee Guida di Sviluppo

- La programmazione parallela richiede il coordinamento attivo ed esplicito dei core attraverso le opportune API di programmazione.
- A causa della maggiore complessità rispetto ai programmi sequenziali, lo sviluppo di software parallelo necessita di adeguate tecniche formali di progettazione ed è indissociabile da una profonda conoscenza dell'hardware su cui il programma verrà eseguito.

Programmazione a Memoria Distribuita con MPI: Fondamenti e Tecniche

1. Architettura Distribuita e Modello SPMD
Nei sistemi a memoria distribuita ogni nodo/processore possiede la propria memoria locale privata. Non essendoci uno spazio di indirizzamento condiviso, i processi comunicano esclusivamente tramite scambio di messaggi (*message passing*) sulla rete di interconnessione. Il paradigma di riferimento è **SPMD** (*Single-Program Multiple-Data*): viene compilato un unico file eseguibile eseguito in parallelo da molteplici processi, il cui comportamento specifico viene ramificato tramite strutture condizionali (`if-else`) basate sull'identificativo univoco (*rank*) del processo.

2. Struttura di un Programma MPI, Compilazione, Esecuzione e Debugging
I programmi MPI in C includono l'header `<mpi.h>`. Gli identificatori definiti da MPI iniziano con `MPI_`, con prima lettera maiuscola dopo l'underscore per funzioni e tipi di dato.

- **Inizializzazione e Chiusura**: La funzione `MPI_Init(&argc, &argv)` inizializza l'ambiente MPI e deve precedere qualsiasi altra chiamata MPI. La routine `MPI_Finalize()` libera le risorse allocate ed individua la fine delle chiamate MPI.
- **Compilazione ed Esecuzione**: La compilazione sfrutta lo script wrapper `mpicc` (es. `mpicc -g -Wall -o mpi_hello mpi_hello.c`). L'esecuzione avviene tramite `mpiexec -n <num_procs> <eseguibile>`.
- **Debugging Parallelo**: Poiché esaminare lo stato di più processi è complesso, è possibile avviare uno specifico processo sotto debugger (es. `gdb` o `ddd`) combinando le opzioni in `mpiexec` (es. `mpiexec -n 4 ./test : -n 1 ddd ./test : -n 1 ./test`).

3. Comunicatori e Identificazione dei Processi
Un **comunicatore** è una collezione di processi abilitati allo scambio di messaggi. All'avvio viene creato il comunicatore globale `MPI_COMM_WORLD`, che racchiude tutti i p processi avviati (con rank da 0 a $p-1$).

- `MPI_Comm_size(comm, &comm_sz)`: Restituisce il numero totale di processi nel comunicatore.
- `MPI_Comm_rank(comm, &my_rank)`: Restituisce il rank del processo chiamante.

4. Comunicazione Punto-a-Punto Bloccante

La comunicazione punto-a-punto si avvale di funzioni di invio e ricezione:

- `MPI_Send`: Parametri: buffer dati, numero di elementi (**non** byte), tipo di dato MPI, rank di destinazione, tag del messaggio e comunicatore.
- `MPI_Recv`: Parametri: buffer di destinazione, capacità massima del buffer in numero di elementi, tipo di dato, rank di origine, tag, comunicatore e struttura di stato (`MPI_Status`).
- **Tipi di Dato MPI**: Corrispondono ai tipi nativi C (`MPI_CHAR`, `MPI_INT`, `MPI_FLOAT`, `MPI_DOUBLE`, `MPI_BYTE`, ecc.).
- **Matching e Wildcard**: Il messaggio viene ricevuto con successo se il tipo corrisponde e la capacità del buffer di ricezione è maggiore o uguale alla dimensione del messaggio inviato. Gli invii tra la stessa coppia di processi mantengono l'ordine di partenza (*non-overtaking*). Il ricevitore può usare le wildcard `MPI_ANY_SOURCE` e `MPI_ANY_TAG`.
- **Informazioni sullo Stato**: Usando le wildcard, il mittente ed il tag effettivi si ricavano da `status.MPI_SOURCE` e `status.MPI_TAG`. La quantità esatta di elementi ricevuti si ottiene mediante `MPI_Get_count(&status, datatype, &count)`.

5. Semantica di `MPI_Send`, Protocolli Interni e Performance

Il completamento di `MPI_Send` garantisce unicamente la riutilizzabilità del buffer di invio, non la consegna o la ricezione del messaggio. MPI sceglie trasparentemente tra due protocolli sulla base di una soglia dimensionale (*eager threshold*):

- **Eager Protocol (Messaggi Piccoli)**: Il messaggio viene inviato senza un handshake preventivo e copiato in un buffer gestito da MPI. Garantisce latenza minima ed un ritorno immediato di `MPI_Send`, ma comporta fino a due copie lato CPU ed occupazione di memoria sul ricevente (*unexpected queue*).
- **Rendezvous Protocol (Messaggi Grandi)**: Prevede un handshake esplicito con invio di un messaggio di controllo RTS (*Request To Send*) ed attesa del riscontro CTS (*Clear To Send*) dal destinatario. Non richiede copie aggiuntive (trasferimento zero-copy via DMA/RDMA direttamente tra i buffer utente), ma introduce la latenza di un *round-trip* e blocca il mittente fino all'invocazione di `MPI_Recv`.

- **Modello di Costo ed Efficienza**: Il tempo di trasferimento è modellato come $T(n) = \alpha + n/\beta$ (dove α è la latenza e β la larghezza di banda). Per ottimizzare il codice occorre raggruppare i dati ed inviare un numero minore di messaggi di grande dimensione anziché molti messaggi piccoli.

- **Livello di Rete (TCP vs RDMA)**: Reti classiche TCP/IP su Ethernet richiedono chiamate di sistema, gestione delle interruzioni e copie nel kernel. Architetture ad alte prestazioni (InfiniBand, RoCE, Slingshot) supportano **RDMA** (*Remote Direct Memory Access*), eliminando l'intervento del kernel ed azzerando l'overhead sulla CPU.

6. Modalità di Comunicazione Punto-a-Punto

Oltre alla modalità Standard (`MPI_Send`), MPI fornisce:

- **Buffered Mode (`MPI_Bsend`)**: L'invio è sempre localmente bloccante e ritorna non appena i dati sono copiati in un buffer esplicitamente allocato dall'utente.

- **Synchronous Mode (`MPI_Ssend`)**: Ritorna solo quando il destinatario ha iniziato la ricezione del messaggio (bloccante globale).

- **Ready Mode (`MPI_Rsend`)**: Può essere chiamata solo se la corrispondente `MPI_Recv` è già stata avviata, riducendo l'overhead dell'handshake.

7. Comunicazione Non Bloccante

Per sovrapporre computazione e comunicazione (*overlap*) e prevenire deadlock si utilizzano le funzioni *immediate*:

- **Invocazione**: `MPI_Isend` e `MPI_Irecv` si avviano senza bloccare l'esecuzione e restituiscono un handle `MPI_Request`.

- **Sincronizzazione**: La verifica del completamento avviene tramite `MPI_Wait` (bloccante, invalida l'handle a `MPI_REQUEST_NULL`) o `MPI_Test` (non bloccante, restituisce un flag di stato). Per gestire array di richieste si usano `MPI_Waitall`, `MPI_Waitany`, `MPI_Testall`, `MPI_Testany`.

8. Deadlock ed Esercizi Applicativi

Presupporre erroneamente che `MPI_Send` sia sempre non bloccante causa deadlock se due processi si inviano reciprocamente dati con il protocollo Rendezvous prima di eseguire le rispettive `MPI_Recv`.

Esercizio 1: Hello World Ordinato

I processi con $rank > 0$ inviano il proprio messaggio di saluto al $rank$ 0. Il $rank$ 0 stampa il proprio messaggio e poi raccoglie in ordine sequenziale (con un ciclo da $q = 1$ a $comm_sz - 1$) i messaggi tramite `MPI_Recv`, garantendo l'output ordinato.

Esercizio 2: Topologia ad Anello (Ring Topology)

Ogni processo calcola l'indice dei vicini $prev = (rank - 1 + numtasks)\%numtasks$ e

$next = (rank + 1)\%numtasks$. L'invio e la ricezione bidirezionale si implementano in modo sicuro mediante chiamate non bloccanti (`MPI_Irecv` e `MPI_Isend`) per entrambi i vicini, concluse da `MPI_Waitall`:

```
MPI_Irecv(&buf[0], 1, MPI_INT, prev, 0, MPI_COMM_WORLD, &reqs[0]);
MPI_Irecv(&buf[1], 1, MPI_INT, next, 0, MPI_COMM_WORLD, &reqs[1]);
MPI_Isend(&rank, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, &reqs[2]);
MPI_Isend(&rank, 1, MPI_INT, next, 0, MPI_COMM_WORLD, &reqs[3]);
MPI_Waitall(4, reqs, stats);
```

Esercizio 3: Calcolo dell'Integrale (Regola dei Trapezi Parallela)
L'approssimazione dell'integrale di $f(x)$ su $[a, b]$ tramite n trapezi di ampiezza $h = (b - a)/n$ ha formula seriale:

$$\text{Area} = h \left[\frac{f(a)}{2} + \sum_{i=1}^{n-1} f(x_i) + \frac{f(b)}{2} \right]$$

Nella versione parallela:

1. Ogni processo calcola $local_n = n/comm_sz$.
2. Definisce l'intervallo locale con $local_a = a + my_rank \cdot local_n \cdot h$ e $local_b = local_a + local_n \cdot h$.
3. Computa $local_integral = Trap(local_a, local_b, local_n, h)$.
4. I processi con $rank \neq 0$ inviano $local_integral$ al $rank$ 0, il quale somma i contributi parziali e visualizza il risultato finale.

Esercizio 4: Stima del π mediante Metodo Monte Carlo
Ipotizzando di lanciare dardi in modo uniforme su un quadrato di lato 2 ($[-1, 1] \times [-1, 1]$), il rapporto tra dardi interni al cerchio iscritto ($r = 1$, equazione $x^2 + y^2 \leq 1$) e lanci totali approssima $\frac{\pi}{4}$. Nel programma MPI, ogni processo genera casualmente un sottoinsieme dei lanci complessivi e determina quanti punti soddisfano $x^2 + y^2 \leq 1$. I conteggi parziali vengono trasmessi al rank master, che calcola la stima finale $\pi \approx 4 \times \frac{\text{totale_interni}}{\text{totale_lanci}}$.

7 Fondamenti di Programmazione Distribuita con MPI

1. Architettura Hardware e Modello di Programmazione SPMD

Nei sistemi a memoria distribuita, ciascun nodo di elaborazione possiede una propria CPU e una memoria locale privata . Non essendoci memoria condivisa, i vari nodi

comunicano esclusivamente attraverso il passaggio di messaggi su una rete di interconnessione (*interconnect*) .

Il modello software adottato è lo **SPMD** (*Single-Program Multiple-Data*) :

- Viene compilato un singolo sorgente eseguibile .
- Più processi eseguono simultaneamente lo stesso codice .
- Il comportamento specifico di ciascun processo viene differenziato tramite strutture condizionali (`if-else`) basate sull'identificativo unico del processo (*rank*) .

2. Struttura di un Programma MPI, Compilazione ed Esecuzione

I programmi MPI in linguaggio C richiedono l'inclusione dell'header `<mpi.h>` . Gli identificatori e le funzioni definite dallo standard iniziano con il prefisso `MPI_`, con la prima lettera dopo il trattino basso in maiuscolo (es. `MPI_Init`) .

- `MPI_Init(int* argc, char*** argv)`: Inizializza l'ambiente di esecuzione MPI. Nessuna chiamata MPI può precedere questa funzione .
- `MPI_Finalize(void)`: Libera le risorse allocate da MPI e conclude l'ambiente parallelo. Nessuna chiamata MPI può seguirla .
- **Compilazione**: Si utilizza il wrapper del compilatore: `mpicc -g -Wall -o mpi_hello mpi_hello.c` .
- **Esecuzione**: Si usa il runner di processo: `mpiexec -n <num_procs> ./mpi_hello` .
- **Debugging**: Il debugging di programmi paralleli è complesso a causa dello stato distribuito dei processi; è possibile collegare debugger come `gdb` o `ddd` a un singolo processo lanciando istanze dedicate tramite `mpiexec` .

3. Processi e Comunicatori

- **Rank**: Ogni processo è identificato all'interno di un comunicatore da un numero intero non negativo compreso tra 0 e $p-1$ (dove p è il numero totale di processi) .
- **Comunicatore**: Un gruppo di processi abilitati a scambiarsi messaggi . `MPI_COMM_WORLD` è il comunicatore predefinito creato all'avvio, contenente tutti i processi istanziati .
- `MPI_Comm_size(MPI_Comm comm, int* size)`: Restituisce il numero totale di processi nel comunicatore .

- `MPI_Comm_rank(MPI_Comm comm, int* rank)`: Restituisce il *rank* del processo chiamante .

- **Non-determinismo**: L'ordine di stampa dell'output su console da parte di più processi non è garantito ed è indeterminato .

4. Comunicazione Point-to-Point Bloccante

Le operazioni fondamentali per lo scambio di dati tra due processi specifici sono:

- `MPI_Send(void* buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)` .

- `MPI_Recv(void* buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status* status)` .

Proprietà chiave e regole di corrispondenza:

- I parametri `count` specificano il **numero di elementi**, non la dimensione in byte .

- I tipi di dato MPI (`MPI_INT`, `MPI_FLOAT`, `MPI_DOUBLE`, `MPI_CHAR`, `MPI_BYTE`, ecc.) mappano direttamente i tipi C nativi .

- **Regola Non-overtaking**: Se un processo q invia due messaggi consecutivi al processo r, il primo messaggio sarà reso disponibile a r prima del secondo . Non vi sono invece vincoli d'ordine su messaggi inviati da processi differenti .

- **Matching**: Un messaggio viene ricevuto con successo se corrispondono il tipo di dato (`recv_type == send_type`), il comunicatore, la sorgente e il *tag* . La dimensione del buffer di ricezione deve essere maggiore o uguale a quella inviata .

- **Wildcard**: In ricezione è possibile accettare messaggi da qualsiasi mittente con `MPI_ANY_SOURCE` o con qualsiasi etichetta con `MPI_ANY_TAG` .

- **Ispezione dello stato**: Il parametro `status` contiene campi come `MPI_SOURCE`, `MPI_TAG` ed `MPI_ERROR` . La funzione `MPI_Get_count(MPI_Status* status, MPI_Datatype datatype, int* count)` consente di determinare il numero effettivo di elementi ricevuti .

5. Protocolli Trasportistici, Prestazioni e Architetture di Rete

Al ritorno di una chiamata a `MPI_Send`, l'unica garanzia fornita dallo standard è che il buffer utente può essere riutilizzato o modificato; **non** vi è alcuna garanzia che il

messaggio sia stato effettivamente consegnato al destinatario o che sia uscito dal nodo
.

L'implementazione MPI seleziona automaticamente due protocolli principali in base alle dimensioni del messaggio :

1. **Eager Protocol (Messaggi piccoli)**: Il messaggio viene inviato immediatamente e copiato in un buffer intermedio gestito da MPI sul lato ricevente . *Vantaggi*: Latenza minima (nessun handshake), invio rapido . *Svantaggi*: Richiede fino a due copie in memoria e occupa spazio nei buffer del ricevitore .

2. **Rendezvous Protocol (Messaggi grandi)**: Viene inviato prima un messaggio di controllo RTS (*Request To Send*). Il mittente rimane in attesa di una risposta CTS (*Clear To Send*) inviata dal ricevente soltanto dopo l'attivazione della rispettiva `MPI_Recv` . *Vantaggi*: Trasferimento diretto da buffer a buffer utente (**Zero-copy**), nessun consumo di memoria extra . *Svantaggi*: Introduce il costo di un *round-trip* di rete e forza il mittente ad attendere il ricevitore .

Prestazioni e Modello di Trasmissione: Il tempo di trasferimento di un messaggio di dimensione n è modellato dalla formula $T(n) = \alpha + \frac{n}{\beta}$, dove α rappresenta la latenza di rete/overhead software e β è la larghezza di banda . Per ottimizzare le prestazioni, è altamente preferibile accorpare i dati ed inviare pochi messaggi di grandi dimensioni per ridurre l'impatto di α .

Sotto-livello di Rete (TCP vs RDMA):

- **TCP/IP (Ethernet classico)**: Richiede copie nel kernel, interruzioni della CPU e gestione dello stack TCP, causando latenza elevata e carico CPU .

- **RDMA (InfiniBand, RoCE, Slingshot)**: Consente alla scheda di rete (NIC) di accedere direttamente alla memoria dello spazio utente registrata, azzerando le copie, scavalcando il kernel e liberando la CPU da compiti di trasferimento .

Stallo (Deadlock): Se due processi eseguono simultaneamente una `MPI_Send` bloccante l'uno verso l'altro prima di invocare `MPI_Recv`, il programma va in *deadlock* se la dimensione del messaggio supera la soglia del protocollo Eager (*eager threshold*)
.

6. Comunicazione Non Bloccante e Modalità di Invio

Per sovrapporre computazione e comunicazione (*overlap*) ed evitare deadlock, MPI fornisce varianti non bloccanti (*immediate*) :

- `MPI_Isend(void* buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm, MPI_Request* request)` .

- `MPI_Irecv(void* buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Request* request)` .

Queste funzioni restituiscono immediatamente un handle `MPI_Request` . L'operazione deve poi essere completata esplicitamente tramite:

- `MPI_Wait(MPI_Request* request, MPI_Status* status)`: Chiamata bloccante che attende la fine dell'operazione e invalida l'handle .
- `MPI_Test(MPI_Request* request, int* flag, MPI_Status* status)`: Controllo non bloccante che imposta `flag = 1` se l'operazione è completata .
- Varianti per multipli handle: `MPI_Waitall`, `MPI_Testall`, `MPI_Waitany`, `MPI_Testany` .

Modalità di Invio Point-to-Point:

1. **Standard Mode (`MPI_Send`)**: Gestione automatica tra buffering locale (Eager) o sincronizzazione (Rendezvous) .
2. **Buffered Mode (`MPI_Bsend`)**: L'invio usa un buffer allocato dall'utente; la chiamata ritorna non appena i dati sono copiati nel buffer .
3. **Synchronous Mode (`MPI_Ssend`)**: Bloccante a livello globale; ritorna solo dopo che il ricevitore ha avviato la `MPI_Recv` .
4. **Ready Mode (`MPI_Rsend`)**: Ritorna con successo solo se la `MPI_Recv` corrispondente è già stata aperta sul ricevitore, riducendo l'overhead dell'handshake .

7. Pattern Applicativi ed Esercizi Pratici

- **Stampa Ordinata (Hello World v2)**: Ciascun processo $1 \dots p-1$ compone il proprio messaggio ed esegue una `MPI_Send` verso il rank 0. Il rank 0 esegue un ciclo for da 1 a $p-1$ ricevendo ed stampando in sequenza i messaggi ordinati .
- **Topologia ad Anello (Ring)**: Ogni processo comunica con i vicini logici `prev = (rank - 1 + size) % size` e `next = (rank + 1) % size` . L'uso di `MPI_Irecv` e `MPI_Isend` abbinato a `MPI_Waitall` previene deadlock durante le fasi di scambio incrociato dei dati .
- **Integrazione con la Regola dei Trapezi**: Calcolo dell'integrale definito nell'intervallo $[a, b]$ suddividendo l'area in n trapezi di ampiezza $h = \frac{b-a}{n}$. L'intervallo globale viene ripartito equamente assegnando a ciascun processo un sotto-intervallo $[a_{\text{local}}, b_{\text{local}}]$ e $n_{\text{local}} = \frac{n}{p}$ trapezi . Ciascun processo calcola il proprio integrale locale `local_integral` ; i processi con `rank != 0` inviano il risultato al rank 0, che somma i contributi locali ed esegue la riduzione complessiva .

- **Approssimazione di π con Metodo Monte Carlo**: Generando punti casuali (x, y) uniformemente distribuiti all'interno di un quadrato di lato 2 centrato nell'origine (intervallo $[-1, 1]$), e contando quanti cadono all'interno del cerchio di raggio $r = 1$ ($x^2 + y^2 \leq 1$), si calcola la stima :

$$\frac{\text{punti nel cerchio}}{\text{lanci totali}} \approx \frac{\pi}{4} \implies \pi \approx 4 \times \frac{\text{punti nel cerchio}}{\text{lanci totali}}$$

  Il carico dei lanci totali viene suddiviso tra i processi ed i conteggi parziali vengono aggregati per determinare la stima finale .

8 Comunicazioni Collettive in MPI (MPI Collectives)

8.1 1. Richiamo e Limitazioni delle Comunicazioni Point-to-Point

Nelle architetture a memoria distribuita basate su `MPI`, le comunicazioni punto-a-punto (`MPI_Send`, `MPI_Recv`, le varianti non bloccanti `MPI_Isend`, `MPI_Irecv` e le modalità `Bsend`, `Ssend`, `Rsend`) consentono lo scambio diretto di dati tra due singoli processi. Tuttavia, il coordinamento di algoritmi paralleli su larga scala mediante sole comunicazioni punto-a-punto introduce significativi problemi di efficienza e bilanciamento del carico:

- **Calcolo della Somma Globale Ingenuo (es. Regola dei Trapezi):** Se $p - 1$ processi inviano i propri risultati parziali direttamente al processo 0 per la somma finale, il processo 0 diventa un collo di bottiglia. Il tempo richiesto sul processo 0 è pari a:

$$T_0 = (p - 1) \cdot (T_{\text{sum}} + T_{\text{recv}})$$

  mentre gli altri processi terminano in tempo T_{send}, generando un forte sbilanciamento.

- **Riduzione ad Albero Binario:** Organizzando la comunicazione ad albero, il tempo di esecuzione sul processo 0 scende a:

$$T_0 = \log_2(p) \cdot (T_{\text{sum}} + T_{\text{recv}})$$

- **Ottimalità Dipendente dall'Architettura:** L'albero di comunicazione ottimale dipende dal numero di processi p, dalla dimensione dei dati e dalle caratteristiche fisiche del sistema (topologia della rete, numero di schede NIC). Da qui la necessità di primitive collettive native ottimizzate direttamente dalla libreria `MPI`.

8.2 2. Principi Fondamentali e Regole Tassative (*Caveats*) delle Collettive

Una comunicazione collettiva coinvolge **tutti** i processi appartenenti a un determinato comunicatore (`MPI_Comm`). Le regole tassative per il loro corretto utilizzo sono:

1. **Invocazione Unanime:** Tutti i processi nel comunicatore devono chiamare la medesima funzione collettiva. Tassativamente vietato accoppiare una chiamata collettiva (es. `MPI_Reduce`) con una punto-a-punto (es. `MPI_Recv`): tale errore causa stallo (*deadlock*) o crash del programma.

2. **Compatibilità degli Argomenti:** Gli argomenti passati da ciascun processo devono essere coerenti. Ad esempio, il parametro del processo radice/destinazione (`root/dest_process`) deve coincidere su tutti i rank.

3. **Obbligatorietà dei Puntatori di Ricezione:** L'argomento del buffer di uscita (`recvbuf/output_data_p`) produce effetti solo sul processo di destinazione, ma **tutti** i processi devono comunque passare un argomento valido (eventualmente un puntatore non nullo o `NULL` se consentito, ma il parametro non può essere omesso).

4. **Assenza di Tag e Matching Sequenziale:** Le comunicazioni collettive **non** utilizzano i tag `MPI`. L'accoppiamento (*matching*) avviene esclusivamente in base al comunicatore e all'**ordine temporale rigoroso** in cui le funzioni vengono invocate dai vari processi. Se i processi invertono l'ordine di invocazione di due collettive, si verificano errori irreversibili di corruzione dei dati.

8.3 3. Primitive Collettive Fondamentali

8.3.1 MPI_Reduce

Combina i dati forniti da ciascun processo mediante un'operazione aritmetica/logica e deposita il risultato finale sul solo processo radice (`root`).

- **Firma C:**

```
int MPI_Reduce(const void* sendbuf, void* recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, int root,
               MPI_Comm comm);
```

- **Operatori Predefiniti (`MPI_Op`):**
  - `MPI_MAX`, `MPI_MIN`: Massimo e minimo valore.
  - `MPI_SUM`, `MPI_PROD`: Somma e prodotto.
  - `MPI_LAND`, `MPI_BAND`: AND logico e bitwise.
  - `MPI_LOR`, `MPI_BOR`: OR logico e bitwise.
  - `MPI_LXOR`, `MPI_BXOR`: XOR logico e bitwise.
  - `MPI_MAXLOC`, `MPI_MINLOC`: Valore massimo/minimo e relativo rank d'origine.
- È possibile definire operatori personalizzati tramite la funzione `MPI_Op_create`.

8.3.2 MPI_Bcast (Broadcast)

Trasmette un blocco di dati presente sul processo radice (`root`) a tutti gli altri processi del comunicatore.

- **Firma C:**

```
int MPI_Bcast(void* buffer, int count, MPI_Datatype datatype,
              int root, MPI_Comm comm);
```

- **Caratteristica:** Il parametro `buffer` funge sia da input (per il rank `root`) che da output (per gli altri rank).
- **Complessità Temporale:** Utilizzando una struttura ad albero, il tempo richiesto è $T = \log_2(p) \cdot T_{\text{send}}$, rispetto a $(p-1) \cdot T_{\text{send}}$ dell'invio sequenziale.

8.3.3 MPI_Allreduce

Esegue concettualmente una `MPI_Reduce` seguita da una `MPI_Bcast`, rendendo il risultato della riduzione disponibile a **tutti** i processi.

- **Firma C:**

```
int MPI_Allreduce(const void* sendbuf, void* recvbuf, int count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm);
```

- **Pattern ad Farfalla (*Butterfly Pattern / Recursive Distance Doubling*):** Anziché eseguire in sequenza le due primitive ($T = 2\log_2(p) \cdot T_{\text{send}}$), l'algoritmo a farfalla scambia e riduce i dati a coppie di processi con distanze via via raddoppiate, riducendo il tempo totale a:

$$T = \log_2(p) \cdot T_{\text{send}}$$

  ottenendo un velocizzazione teorica di $2\times$.

8.4 4. Distribuzione e Raccolta di Vettori: MPI_Scatter e MPI_Gather

8.4.1 MPI_Scatter

Divide un vettore contiguo posizionato sul processo radice in blocchi di pari dimensione e invia il blocco i-esimo al processo i.

- **Firma C:**

```
int MPI_Scatter(const void* sendbuf, int sendcount, MPI_Datatype sendtype,
                void* recvbuf, int recvcount, MPI_Datatype recvtype,
                int root, MPI_Comm comm);
```

- **ATTENZIONE CRITICA:** `sendcount` rappresenta il numero di elementi inviati a **ciascun** singolo processo, **NON** il numero totale di elementi contenuti nel buffer di partenza!
- **Varianti e In-Place:** La variante `MPI_Scatterv` gestisce blocchi di dimensioni differenti. Sul processo radice è possibile utilizzare la costante `MPI_IN_PLACE` come `recvbuf` per evitare duplicazioni di memoria.

8.4.2 MPI_Gather

Operazione duale di `MPI_Scatter`. Raccoglie i blocchi di dati presenti su ciascun processo e li concatena in ordine di rank sul processo radice.

- **Firma C:**

```
int MPI_Gather(const void* sendbuf, int sendcount, MPI_Datatype sendtype,
               void* recvbuf, int recvcount, MPI_Datatype recvtype,
               int root, MPI_Comm comm);
```

- **ATTENZIONE CRITICA:** `recvcount` indica il numero di elementi ricevuti da **ciascun** processo, non la dimensione totale del vettore finale.
- **Esempio di Stampa Distribuita:** Il processo 0 alloca la memoria globale necessaria (`malloc(n * sizeof(double))`) ed invoca `MPI_Gather`, mentre gli altri processi chiamano `MPI_Gather` passando `recvbuf = NULL`.

8.5 5. Layout di Memoria per le Matrici bidimensionali

Il linguaggio C memorizza le matrici in formato *row-major* (righe contigue in memoria). La corretta esecuzione delle comunicazioni collettive su matrici dipende dalla modalità di allocazione:

- **Matrice Statica (`int matrix[N][M]`):** Gli elementi sono occupati in un blocco di memoria unico e contiguo. È possibile trasferire l'intera matrice con una singola chiamata collettiva specificando `count = N * M`.
- **Matrice Dinamica a Doppio Puntatore (`int** a`):** Allocata tramite un array di puntatori alle righe (`a = malloc(N * sizeof(int*))`) e successive allocazioni per le singole righe (`a[i] = malloc(M * sizeof(int))`).

– **ERRORE GRAVE:** Chiamare `MPI_Reduce(a, ...)` oppure `MPI_Reduce(a[0], ..., N*M, ...)` è **errato e causa crash**, poiché le singole righe non sono contigue in memoria.

– **Soluzione Corretta 1 (Ciclo di collettive):**

```
for (int i = 0; i < num_rows; i++) {
    MPI_Reduce(a[i], recvbuf[i], num_cols, MPI_INT, MPI_SUM, 0, comm);
}
```

– **Soluzione Corretta 2 (Allocazione Unidimensionale Contigua):** Allocare un unico blocco `int* a = malloc(N * M * sizeof(int))` e indicizzare gli elementi come `a[i * M + j]`. In questo modo l'intera matrice è contigua e si può usare un'unica operazione collettiva.

8.6 6. Moltiplicazione Matrice-Vettore ($y = A \cdot x$) e MPI_Allgather

8.6.1 Parallelizzazione Standard

Per calcolare il prodotto tra una matrice A ($m \times n$) e un vettore x (n elementi):

Il rank 0 esegue la `MPI_Bcast` del vettore x a tutti i processi.

Il rank 0 esegue la `MPI_Scatter` delle righe di A, assegnando a ciascun processo `local_m` righe.

Ciascun processo calcola i prodotti scalari locali producendo la porzione `local_y`.

Il rank 0 esegue la `MPI_Gather` dei vettori `local_y` per ricostruire il vettore y finale.

8.6.2 Algoritmi Iterativi e MPI_Allgather

Nei metodi iterativi in cui il vettore di output y_k diventa l'input x_{k+1} dell'iterazione successiva ($y_{k+1} = A \cdot y_k$), ogni processo necessita del vettore y completo aggiornato. L'operazione "Gather + Broadcast" viene unificata ed ottimizzata dalla primitiva `MPI_Allgather`.

- **Firma C:**

```
int MPI_Allgather(const void* sendbuf, int sendcount, MPI_Datatype sendtype,
                  void* recvbuf, int recvcount, MPI_Datatype recvtype,
                  MPI_Comm comm);
```

- **Utilizzo:** Ciascun processo mette a disposizione la propria porzione locale `local_x` di dimensione `local_n` e riceve nel buffer destinazione il vettore completo di n elementi.

8.7 7. Collettive Avanzate e Rilevanza Industriale

8.7.1 Altre Collettive di Uso Comune

- **Reduce-Scatter:** Esegue la riduzione elemento per elemento dei vettori di input di tutti i processi, suddivide il vettore risultante in blocchi e ne assegna un singolo blocco a ciascun rank.
- **MPI_Alltoall:** Comunicazione completa "tutti-a-tutti" (*all-to-all personalized communication*). Corrisponde a una trasposizione di blocchi di dati: il processo i invia il blocco j-esimo al processo j.

8.7.2 Applicazioni nel Deep Learning ed HPC

Le operazioni collettive costituiscono la spina dorsale delle prestazioni nel calcolo ad alte prestazioni (HPC) e nell'addestramento distribuito di reti neurali di grandi dimensioni (LLM):

- **Meta FSDP (Fully Sharded Data Parallel):** Architettura di addestramento distribuito che sfrutta ampiamente `MPI_Allgather` per ricostruire i parametri dei livelli prima dei passaggi forward/backward e `Reduce-Scatter` per la sincronizzazione e il partizionamento dei gradienti.
- **Librerie Collettive Vendor-Specific (*CCL):** A causa del peso determinante delle collettive sui tempi trascorsi in comunicazione, i produttori hardware sviluppano librerie altamente specializzate:
  - **NCCL** (NVIDIA Collective Communications Library)
  - **RCCL** (AMD ROCm Communication Collectives Library)
  - **oneCCL** (Intel)
  - **MSCCL** (Microsoft)

  A differenza delle implementazioni MPI generiche (es. Open MPI) che usano euristiche astratte, le librerie `*CCL` ottimizzano gli algoritmi (ring, tree, NVLink-aware) basandosi sulla topologia fisica esatta dell'hardware e dei bus di interconnessione (PCIe, NVLink, NVSwitch).